

1.MyBatis介绍(雷神230集)

MyBatis-和数据库进行交互;持久化层框架

1) 从原始的JDBC----dbutils(QueryRunner)----JdbsTemplate---XXX;称为工具;

工具:一些功能的简单封装

框架:某个领域的整体解决方案; 缓存,考虑异常处理问题,考虑部分字段映射问题...

现状:不用原生JDBC

1. 麻烦;
2. sql语句是硬编码在程序中的;耦合()
3. Hibernate-数据库交互的框架(ORM框架) (使用各种框架)

ORM(Object Relation Mapping)对象关系映射

创建好javaBean;

```
1  @Table("t_employee")
2
3  //session.get("1",Employee.class)
4  class Employee{
5      private Integer empid;
6      private String empName;
7  }
```

优点:

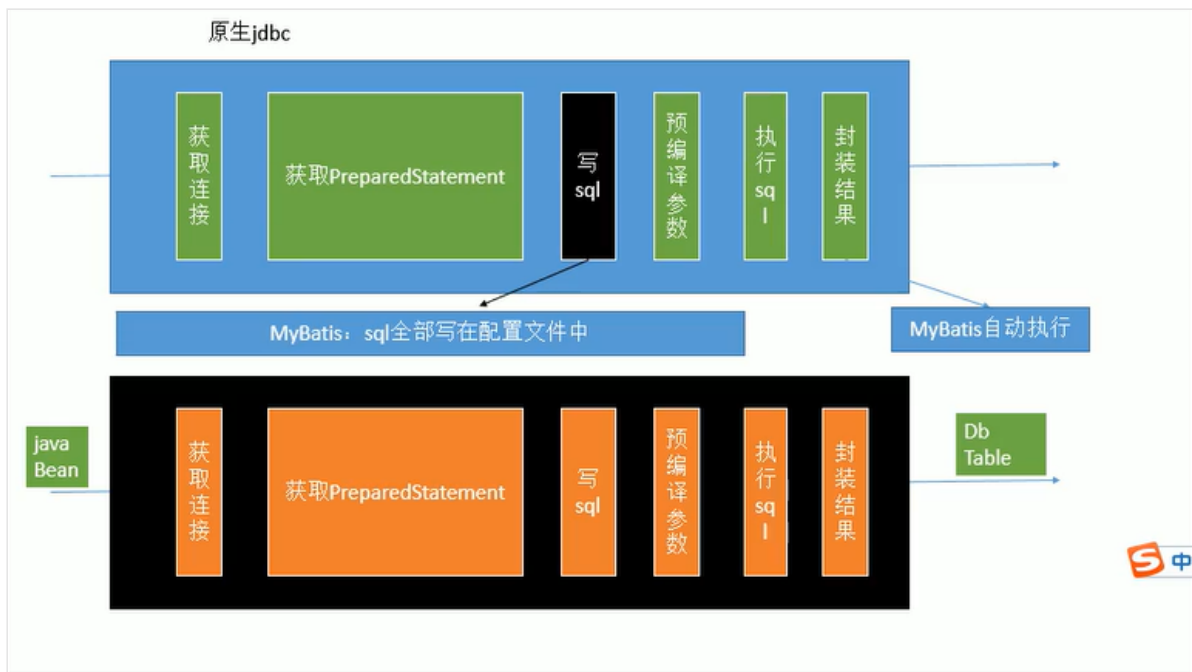
1. 项目启动(数据库没有这张表.Hibernate帮你自动创建)
2. 如果有,且字段更新,Hibernate帮你自动添加字段
3. SQL不用写

缺点: (黑箱半封闭)

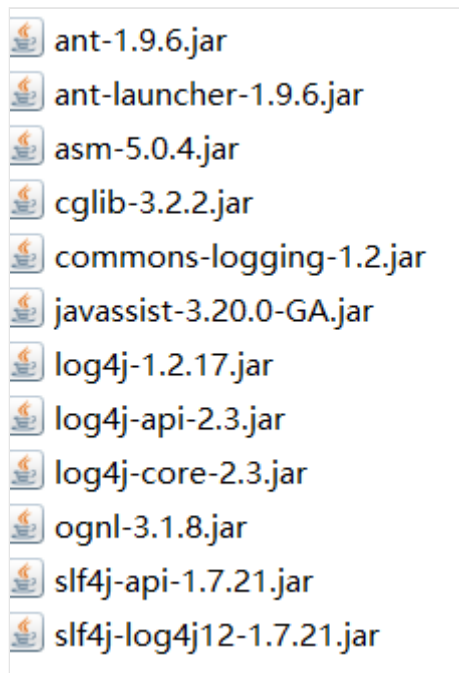
1. 定制SQL;
2. HQL; SQL;
3. 全映射框架; 部分字段映射很难; 能做;

希望:

1. 最好有一个框架能支持定制化SQL,
2. 而且功能强大;
3. SQL也不要硬编码在java文件中,导致维护困难



1. MyBatis将重要的步骤抽取出来可以人工定制,其他步骤自动化;
2. 重要步骤都是写在配置文件中(好维护);
3. 完全解决数据库的优化问题;
4. MyBatis底层就是对原生JDBC的一个简单封装;
5. 既将java编码与sql抽取了出来,还不会失去自动化功能;半自动的持久化层框架
6. 一个轻量级框架



MyBatis-jar包的扩展,需要的话可以自定义导包,但mybatis-3.4.1.jar都已经集成了

2.环境搭建(雷神231-234集)

1. Mybatis_HelloWorld

1. 创建一个java工程
2. 创建测试库,测试表,以及封装数据的javaBean,和操作数据库的dao接口
 1. 创建表:Navicat创建
 2. 创建javaBean: Employee(封装表的数据)
 3. 创建一个Dao接口(EmployeeDao),用来操作数据库;
3. 用MyBatis操作数据库

2. 导包

```
1 //mysql-connector-java-5.1.47.jar
2 //mybatis-3.4.1.jar
3 //建议导入日志包,这样在mybatis关键的环节就会有日志打印
4 //log4j-1.2.17.jar(日志框架)依赖类路径下一个log4j.xml配置文件;
```

3. 写配置

(两个,全局配置文件(指导mybatis运行的),dao接口的实现文件(描述dao中每个方法怎么工作))

1.第一个配置文件:

(成为mybatis的全局配置文件,指导mybatis如何正确运行,比如连接哪个数据库)

2.第二个配置文件:(编写每一个方法都如何向数据库发送sql语句,如何执行...相当于接口的实现类)

- 将mapper的namespace属性改为接口的全类名
- 配置细节

```
1 <!-- namespace:名称空间:写接口的全类名,相当于告诉MyBatis这个配置文件是实现哪个接口的 -->
2 <mapper namespace="com.Ray.dao.EmployeeDao">
3     <!-- public Employee getEmpById(Integer id); -->
4     <!-- select: 用来定义一个查询操作 id:方法名,相当于这个配置是对某个方法的实现
5         resultType:指定方法运行后的返回值类型(查询操作必须指定)
6         #{属性名}:代表取出传递过来的某个参数的值
7     -->
8     <select id="getEmpById" resultType="com.Ray.bean.Employee">
9         select * from t_employee where id = #{id}
10    </select>
11 </mapper>
```

我们写的dao接口的实现文件,mybatis默认是不知道的,需要在全局配置文件中注册

```
1 <!-- 引入我们自己编写的每一个接口的实现文件 -->
2 <mappers>
3     <!-- resource:表示从类路径下找资源(此处的EmployeeDao.xml在conf源文件下也就是在类
4         路径下) -->
5     <mapper resource="EmployeeDao.xml"/>
6 </mappers>
</configuration>
```

4. 测试

根据全局配置文件先创建一个

```
1 //1.根据全局配置文件创建出一个SqlSessionFactory
2 //SqlSessionFactory:是SqlSession工厂,负责创建SqlSession对象;
3 //SqlSession:sql会话(代表和数据库的一次会话)
4 String resource = "mybatis-config.xml";
5 InputStream inputStream = Resources.getResourceAsStream(resource);
6 SqlSessionFactory sqlSessionFactory = new
    SqlSessionFactoryBuilder().build(inputStream);
```

sqlSessionFactory中获取sqlSession对象操作数据库即可

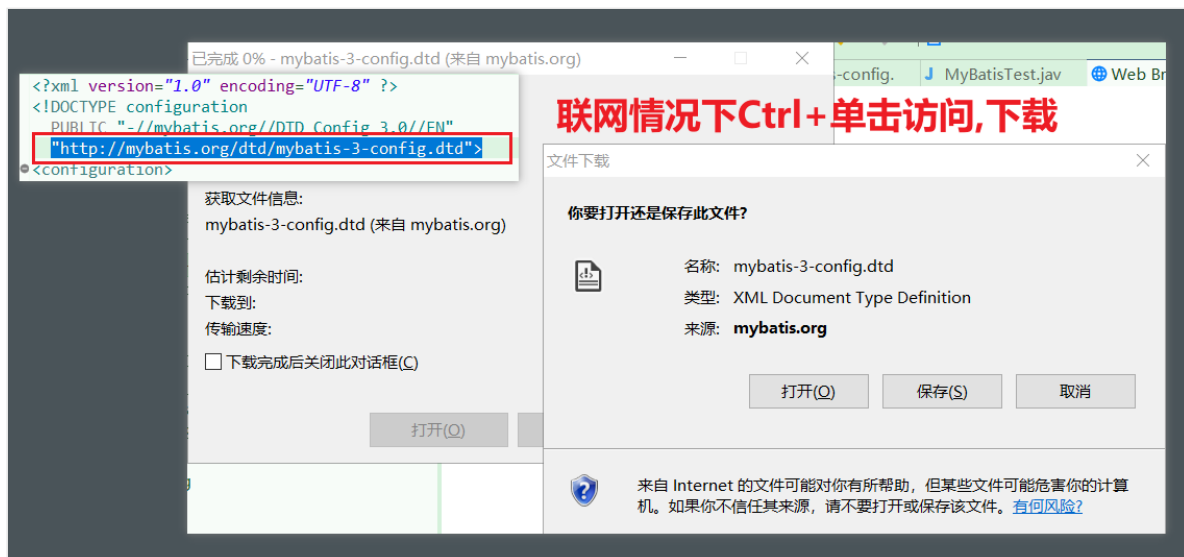
```
1 //2.获取和数据库的一次回话
2 SqlSession openSession = sqlSessionFactory.openSession();
3 //3.使用sqlSession操作数据库,获取到dao接口的实现
4 EmployeeDao employeeDao = openSession.getMapper(EmployeeDao.class);
5 //4.调用之前的方法
6 Employee employee = employeeDao.getEmpById(1);
7 System.out.println(employee);
```



3.如何写xml有提示(项目测试开始)(雷神235集)

1.只要eclipse找到了文件的dtd(约束文件)的位置即可

联网情况下:



未联网情况:

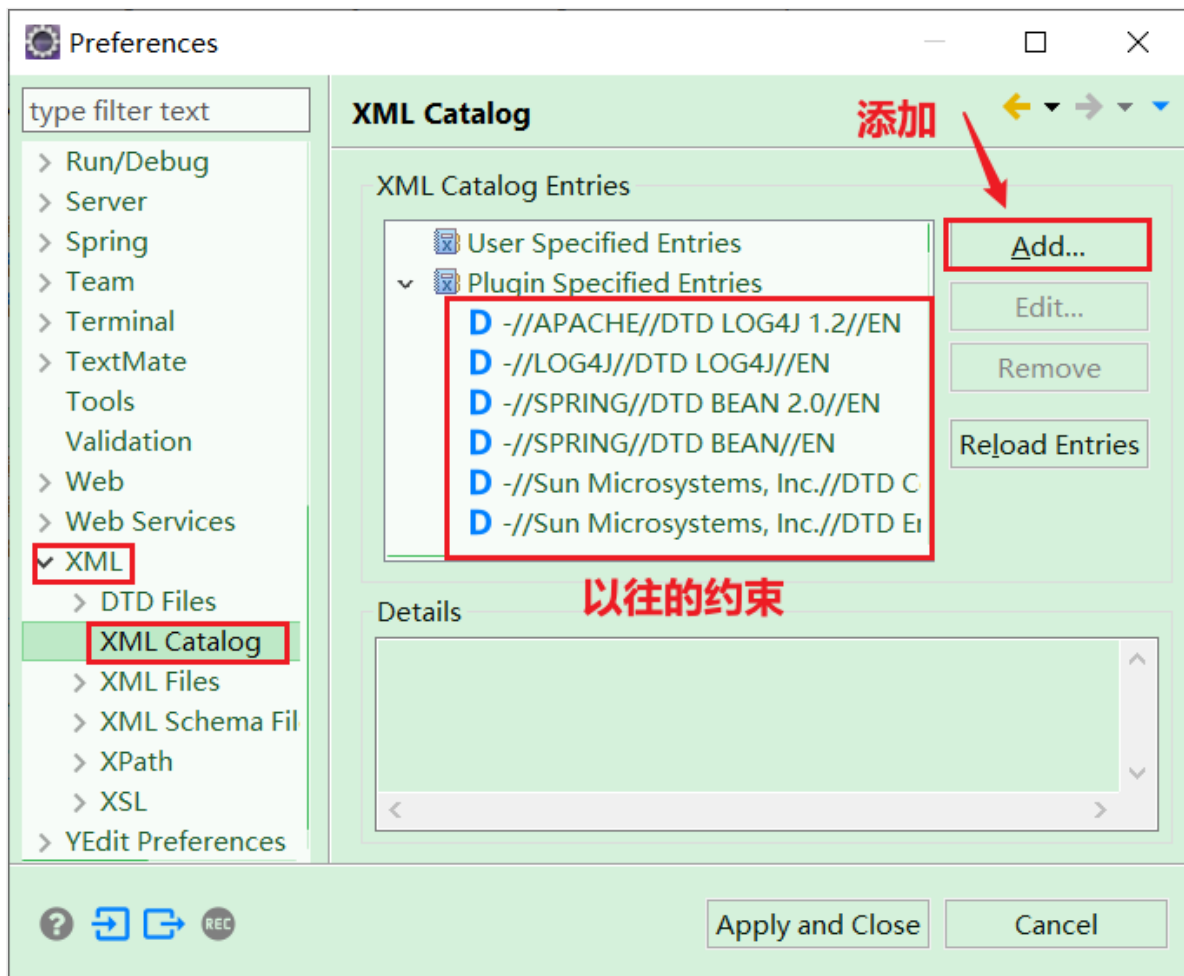
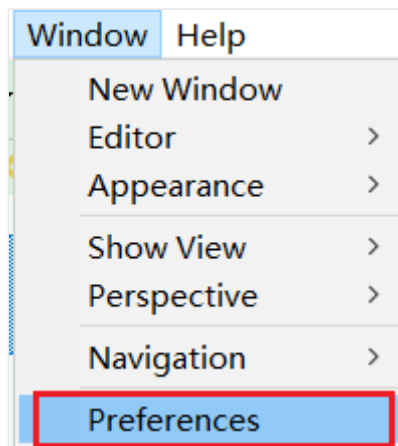


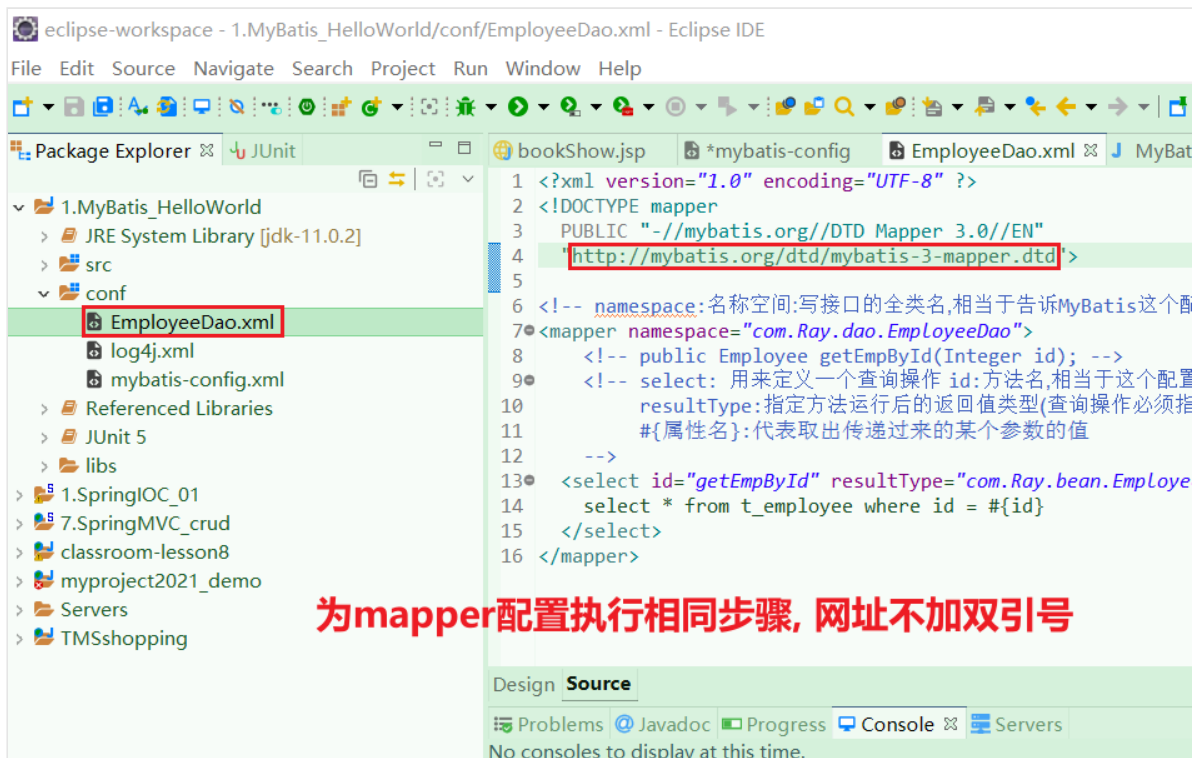
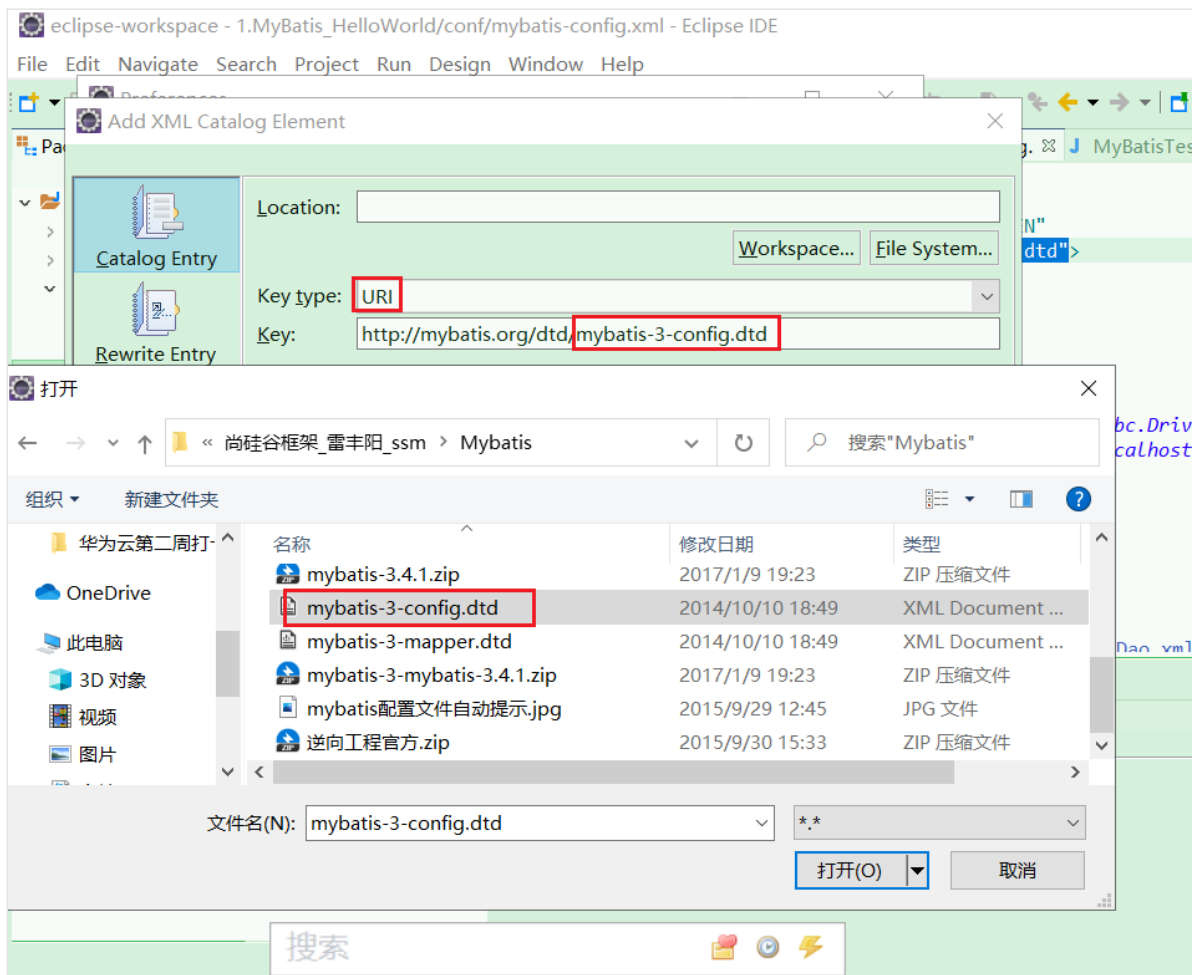
拷贝无效-----从jar包中解压出来

绑定约束文件位置

1.复制dtd的引用网址:<http://mybatis.org/dtd/mybatis-3-config.dtd>

2. 来到window-preference





alt+/不显示的话,关掉配置文件窗口,再重新打开

Tips: 尝试快捷键要在或标签内使用,不然不显示

应该写哪些配置???

```

1 <!-- public Employee getEmpById(Integer id);
2     返回值类型resultType:为全类名;
3     参数类型parameterType:不用写;-->
  
```

```

4      <select id="getEmpById" resultType="com.Ray.bean.Employee">
5          <!-- SQL语句不要写分号 -->
6          select * from t_employee where id = #{id}
7      </select>
8      <!-- public int updateEmployee(Employee employee);
9          返回值类型:增删改不用写,增删改是返回影响多少行
10         参数类型:Mybatis自动判断,如果是数字类型(int long,封装为数字)
11         如果是boolean,影响0行(封装为false,否则就是true)
12         #{属性名},从传入的参数对象中取出对应属性的值
13     -->
14     <update id="updateEmployee" >
15         UPDATE t_employee
16             SET empname=#{empName},gender=#{gender},email=#{email}
17             WHERE id=#{id}
18     </update>

```

1. Mybatis_HelloWorld_Plus(236集)

```

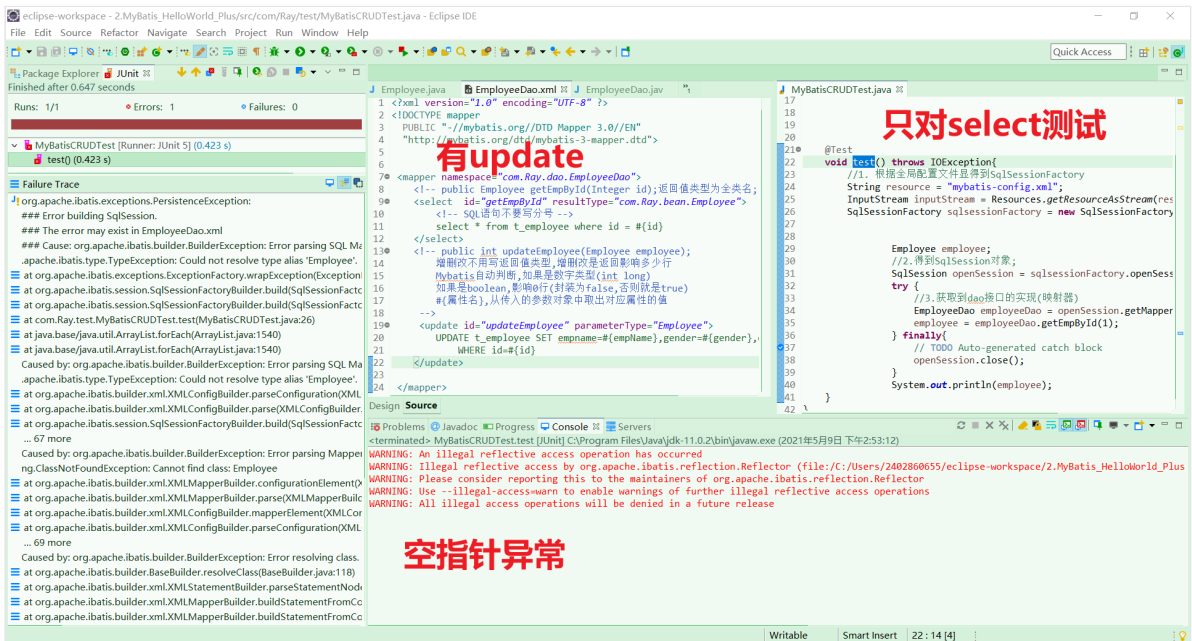
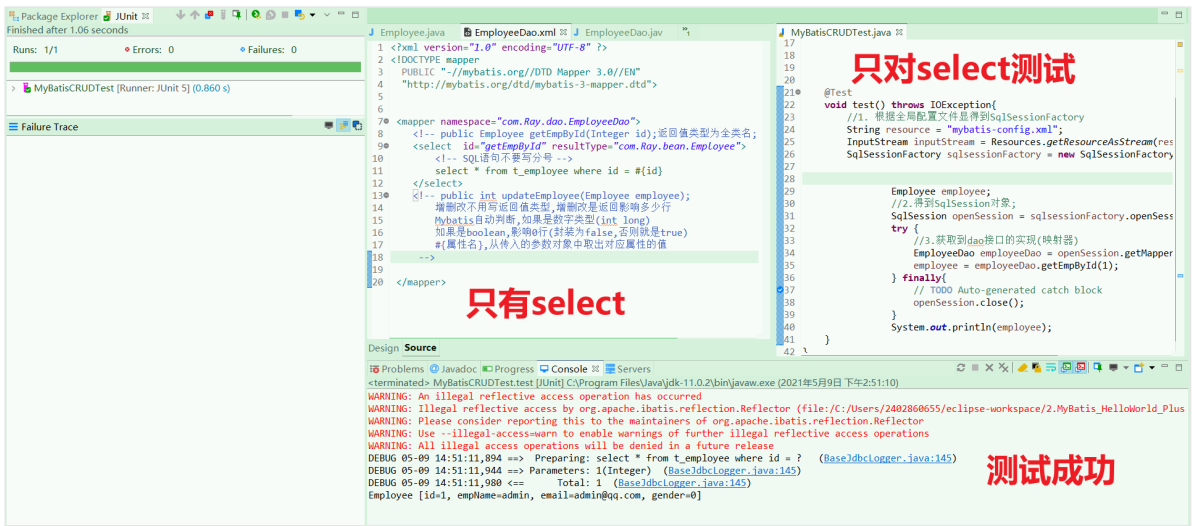
1 package com.Ray.test;
2
3 import java.io.IOException;
4
14 class MyBatisCRUDTest {
15
16     @Test
17     void test() throws IOException {
18         //1. 根据全局配置文件显得到SqlSessionFactory
19         String resource = "mybatis-config.xml";
20         InputStream inputStream = Resources.getResourceAsStream(resource);
21         SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(inputStream);
22         //2.得到SqlSession对象;
23         SqlSession openSession = sqlSessionFactory.openSession();
24         //3.获取到dao接口的实现(映射器)
25         EmployeeDao dao = openSession.getMapper(EmployeeDao.class);
26         try {
27             Employee employee = dao.getEmpById(1);
28             System.out.println(employee);
29         } finally {
30             openSession.close();
31         }
32     }
33 }
34 }
35

```

```

16 class MyBatisCRUDTest {
17     //工厂一个
18     SqlSessionFactory sqlSessionFactory;
19     @Before
20     public void initSqlSessionFactory() throws IOException{
21         String resource = "mybatis-config.xml";
22         InputStream inputStream = Resources.getResourceAsStream(resource);
23         SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(inputStream);
24     }
25     @Test
26     void test() throws IOException {
27         //1. 根据全局配置文件显得到SqlSessionFactory
28         //initSqlSessionFactory此处用了@Before,替代方法调用
29         //2.得到SqlSession对象;
30         SqlSession openSession = sqlSessionFactory.openSession();
31         //3.获取到dao接口的实现(映射器)
32         EmployeeDao dao = openSession.getMapper(EmployeeDao.class);
33         try {
34             Employee employee = dao.getEmpById(1);
35             System.out.println(employee);
36         } finally {
37             openSession.close();
38         }
39     }
40 }
41

```



看雷神视频,注意雷神用的是JUnit4, 用JUnit5需要对应调整

特性	JUnit 4	JUnit 5
在当前类的所有测试方法之前执行。注解在静态方法上。此方法可以包含一些初始化代码。	@BeforeClass	@BeforeAll
在当前类中的所有测试方法之后执行。注解在静态方法上。此方法可以包含一些清理代码。	@AfterClass	@AfterAll
在每个测试方法之前执行。注解在非静态方法上。可以重新初始化测试方法所需要使用的类的某些属性。	@Before	@BeforeEach
在每个测试方法之后执行。注解在非静态方法上。可以回滚测试方法引起的数据库修改。	@After	@AfterEach

测试文件:

```
1 package com.Ray.test;  
2  
3 import java.io.IOException;  
4 import java.io.InputStream;  
5
```

```

6  import org.apache.ibatis.io.Resources;
7  import org.apache.ibatis.session.SqlSession;
8  import org.apache.ibatis.session.SqlSessionFactory;
9  import org.apache.ibatis.session.SqlSessionFactoryBuilder;
10 import org.junit.Before;
11 import org.junit.jupiter.api.BeforeEach;
12 import org.junit.jupiter.api.Test;
13
14 import com.Ray.bean.Employee;
15 import com.Ray.dao.EmployeeDao;
16
17 public class MyBatisCRUDTest {
18     SqlSessionFactory sqlSessionFactory;
19     @BeforeEach
20     public void initSqlSessionFactory() throws IOException {
21         String resource = "mybatis-config.xml";
22         InputStream inputStream = Resources.getResourceAsStream(resource);
23         sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);
24     }
25
26
27
28     @Test
29     void test() throws IOException{
30         //1. 根据全局配置文件得到SqlSessionFactory
31         Employee employee;
32         //2.得到SqlSession对象;
33         SqlSession openSession = sqlSessionFactory.openSession();
34         try {
35             //3. 获取到dao接口的实现(映射器)
36             EmployeeDao employeeDao =
openSession.getMapper(EmployeeDao.class);
37             employee = employeeDao.getEmpById(1);
38         } finally{
39             // TODO Auto-generated catch block
40             openSession.close();
41         }
42         System.out.println(employee);
43     }
44     @Test
45     void insertTest() {
46         //1.获得与数据库的一次会话
47         SqlSession openSession =
sqlSessionFactory.openSession(true); //5.true为自动提交
48         try {
49             //2. 获取到接口的映射器
50             EmployeeDao mapper = openSession.getMapper(EmployeeDao.class);
51             //3.测试
52             int i = mapper.insertEmployee(new Employee(null, "Mike",
"Mike@qq.com", 0));
53             System.out.println(i);
54         } finally{
55             //4.手动提交
56             //openSession.commit();
57             openSession.close();
58         }
59     }

```

```

60     @Test
61     void deleteTest() {
62         SqlSession openSession = sqlSessionSessionFactory.openSession(true);
63         try {
64             EmployeeDao mapper = openSession.getMapper(EmployeeDao.class);
65             boolean b = mapper.deleteEmployee(3);
66             System.out.println(b);
67         } finally{
68             openSession.close();
69         }
70     }
71     @Test
72     void updateTest() {
73         SqlSession openSession = sqlSessionSessionFactory.openSession(true);
74         try {
75             EmployeeDao mapper = openSession.getMapper(EmployeeDao.class);
76             int i = mapper.updateEmployee(new Employee(2, "Tom",
"tom@qq.email", 1));
77             System.out.println(i);
78         } finally{
79             openSession.close();
80         }
81     }
82 }

```

EmployeeDao.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE mapper
3      PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4      "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5  <mapper namespace="com.Ray.dao.EmployeeDao">
6      <!-- public Employee getEmpById(Integer id);返回值类型为全类名；参数类型不用
写;-->
7      <select id="getEmpById" resultType="com.Ray.bean.Employee">
8          <!-- SQL语句不要写分号 -->
9          select * from t_employee where id = #{id}
10     </select>
11     <!-- public int updateEmployee(Employee employee);
增删改不用写返回值类型，增删改是返回影响多少行
Mybatis自动判断，如果是数字类型(int long,自动封装为数字)
如果是boolean,影响0行(封装为false,否则就是true)
#{属性名},从传入的参数对象中取出对应属性的值
-->
12     <update id="updateEmployee">
13         UPDATE t_employee
14             SET empname=#{empName},gender=#{gender},email=#{email}
15             WHERE id=#{id}
16     </update>
17     <!-- public int insertEmployee(Employee employee);-->
18     <insert id="insertEmployee">
19         INSERT INTO t_employee(empName,gender,email)
20         VALUES(#{empName},#{gender},#{email})
21     </insert>
22     <!-- public boolean deleteEmployee(Integer id); -->
23     <delete id="deleteEmployee">
24         DELETE FROM t_employee WHERE id=#{id}
25     </delete>
26
27
28
29

```

```
30 </delete>
31 </mapper>
```

mybatis-config.xml

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE configuration
3   PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
4   "http://mybatis.org/dtd/mybatis-3-config.dtd">
5 <configuration>
6   <environments default="development">
7     <environment id="development">
8       <transactionManager type="JDBC"/>
9       <!-- 配置连接池 -->
10      <dataSource type="POOLED">
11        <property name="driver" value="com.mysql.jdbc.Driver"/>
12        <property name="url"
13          value="jdbc:mysql://localhost:3306/mybatis_0325"/>
14        <property name="username" value="root"/>
15        <property name="password" value="root"/>
16      </dataSource>
17    </environment>
18  </environments>
19  <!-- mapper中注册我们所有写的dao接口的实现文件 -->
20  <mappers>
21    <!-- resource:表示从类路径下找资源(此处的EmployeeDao.xml在conf源文件下也就是在类路
22    径下) -->
23    <mapper resource="EmployeeDao.xml"/>
24  </mappers>
25 </configuration>
```

1.1. 总结:

1. 两个文件:

1. 全局配置文件: mybatis-config.xml; 指导mybatis正确运行的一些全局设置
2. SQL映射文件: EmployeeDao.xml; 相当于是对Dao接口的一个实现描述

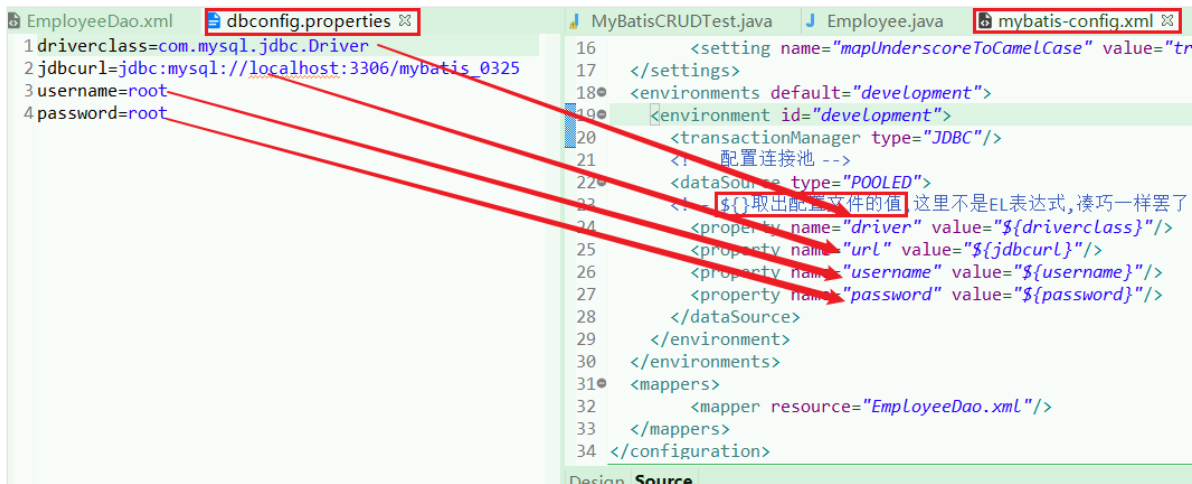
2. 细节

1. 获取到的是接口的代理对象; mybatis自动创建的;
2. SqlSessionFactory和SqlSession;
 1. SqlSessionFactory创建SqlSession对象,Factory只new一次就行
 2. SqlSession: 相当于connection和数据库进行交互的,和数据库的一次回话就应该创建一个新的sqlSession;

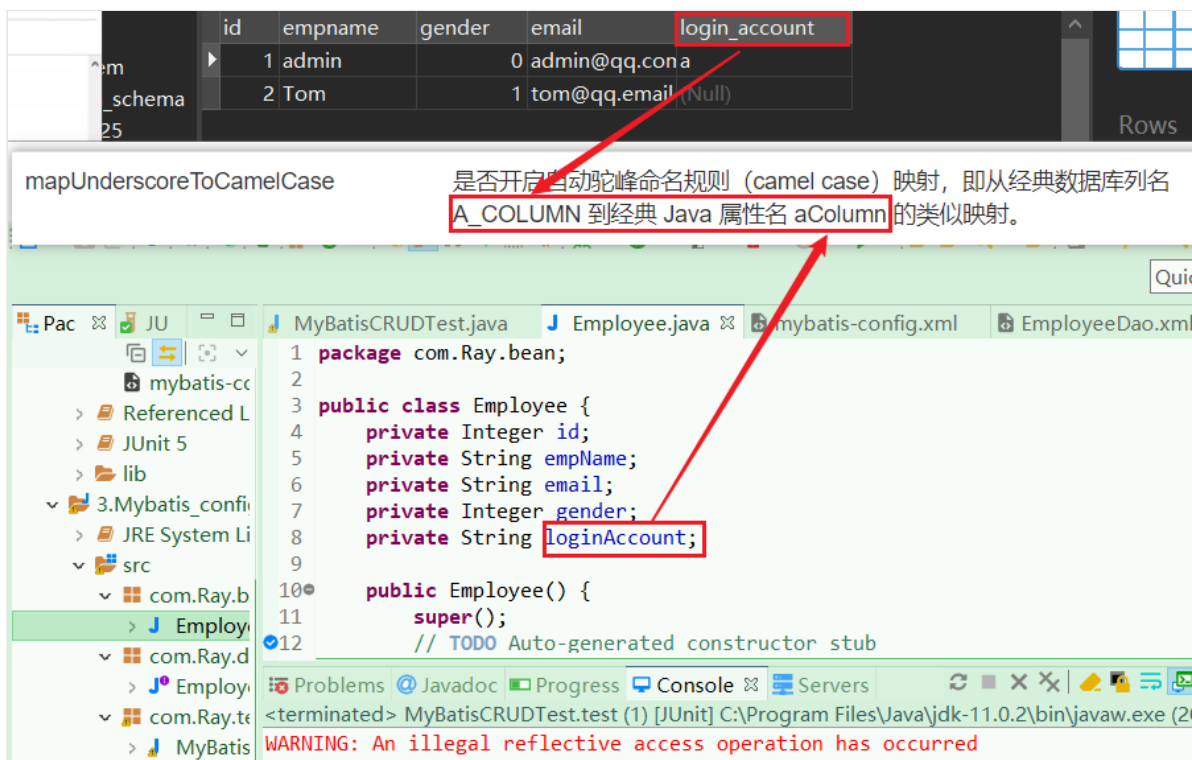
2. 全局配置文件(雷神237-244集)

2.1. configuration 配置(只有mapper和setting重要!!!!)

2.1.1. properties 属性(237集)

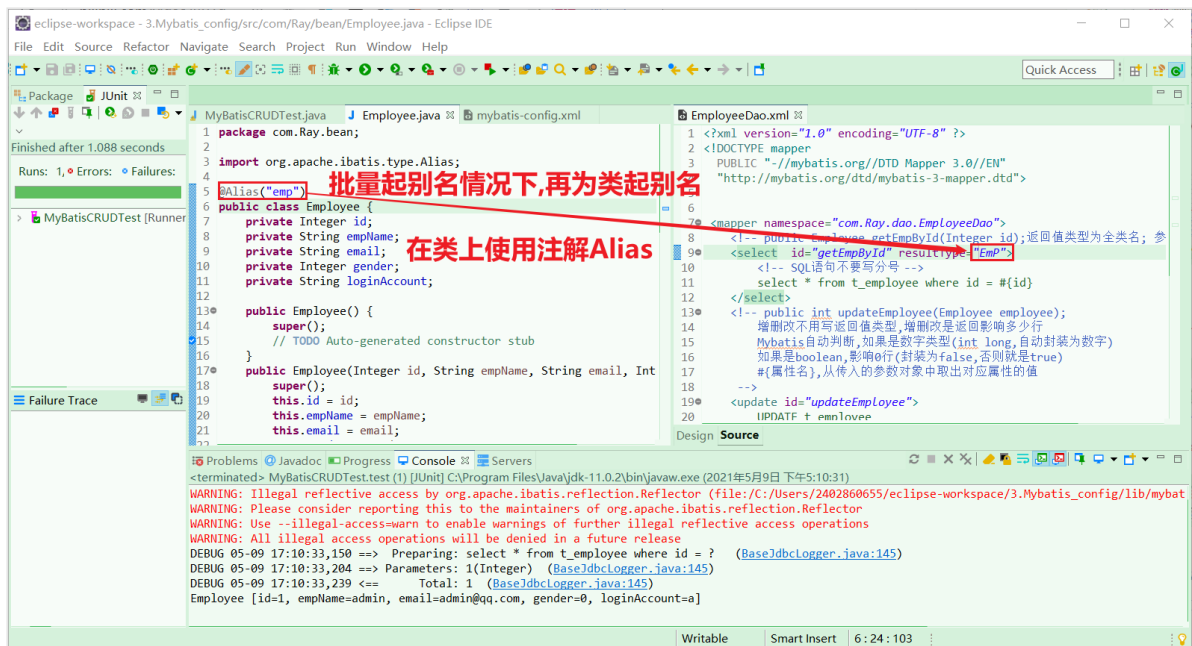
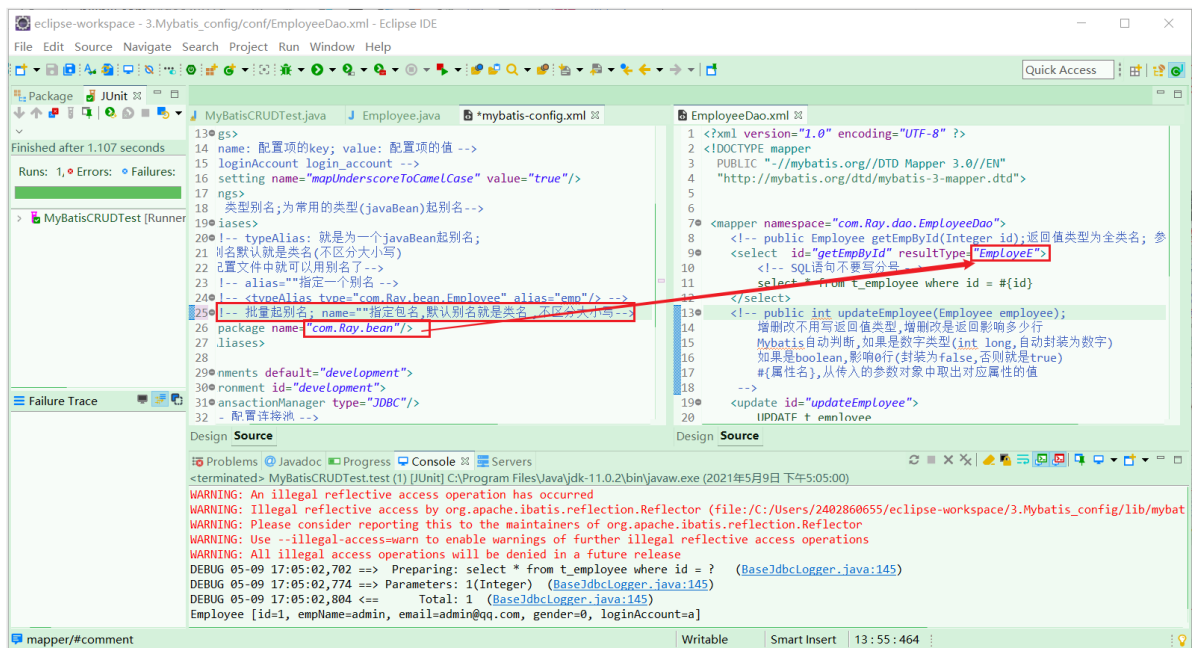
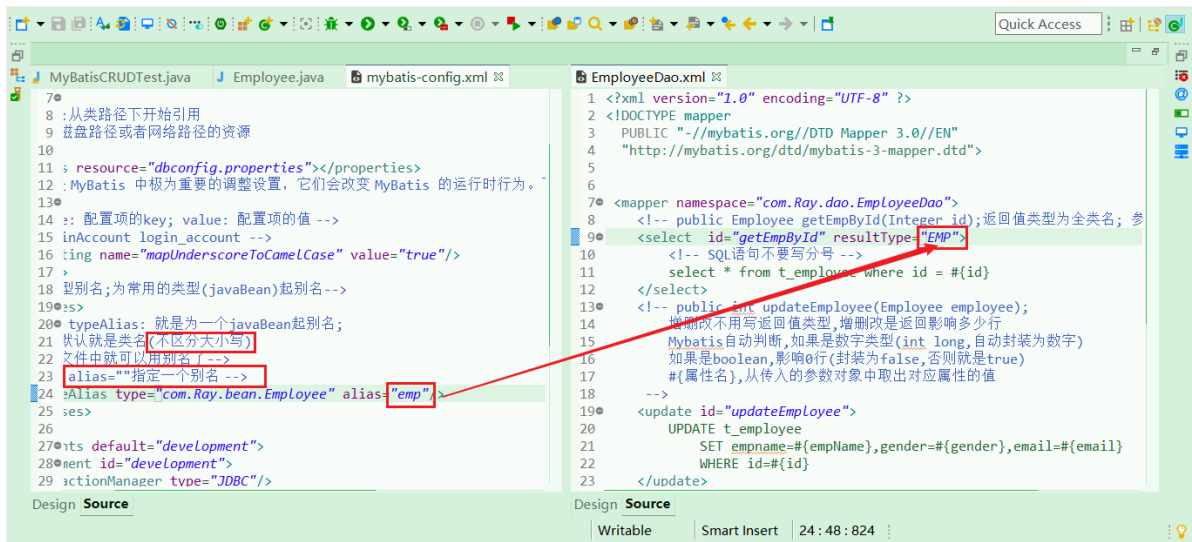


2.1.2. settings 设置(237集)



2.1.3. typeAliases 类型命名(238,239集)





推荐不起别名!!!使用全类名,可以Ctrl+左键,快速进行修改

2.1.4. typeHandlers 类型处理器(240,241集)

原生jdbc执行sql；或者封装结果集

```
Connection connection = dataSource.getConnection();
String sql = "insert into t_employee(empname,email,gender) value(?,?,?)";
PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(0,"admin");
ps.setString(1,"aaa@qq.com");
ps.setInt(2,12);
int i = ps.executeUpdate();
//ps.close();connection.close();
```

typeHandlers

无论是 MyBatis 在预处理语句 (PreparedStatement) 中设置一个参数时, 还是从结果集中取出一个值时, 都会用类型处理器将获取的值以合适的方式转换成 Java 类型。下表描述了一些默认的类型处理器。

NOTE If you use classes provided by JSR-310(Date and Time API), you can use the [mybatis-typehandlers-jsr310](#) .

类型处理器	Java 类型	JDBC 类型
<code>BooleanTypeHandler</code>	<code>java.lang.Boolean, boolean</code>	数据库兼容的 <code>BOOLEAN</code>
<code>ByteTypeHandler</code>	<code>java.lang.Byte, byte</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>BYTE</code>
<code>ShortTypeHandler</code>	<code>java.lang.Short, short</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>SHORT INTEGER</code>
<code>IntegerTypeHandler</code>	<code>java.lang.Integer, int</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>INTEGER</code>
<code>LongTypeHandler</code>	<code>java.lang.Long, long</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>LONG INTEGER</code>
<code>FloatTypeHandler</code>	<code>java.lang.Float, float</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>FLOAT</code>
<code>DoubleTypeHandler</code>	<code>java.lang.Double, double</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>DOUBLE</code>
<code>BigDecimalTypeHandler</code>	<code>java.math.BigDecimal</code>	数据库兼容的 <code>NUMERIC</code> 或 <code>DECIMAL</code>
<code>StringTypeHandler</code>	<code>java.lang.String</code>	<code>CHAR, VARCHAR</code>
<code>ClobReaderTypeHandler</code>	<code>java.io.Reader</code>	-

你可以重写类型处理器或创建你自己的类型处理器来处理不支持的或非标准的类型。具体做法为: 实现 `org.apache.ibatis.type.TypeHandler` 接口, 或继承一个很便利的类 `org.apache.ibatis.type.BaseTypeHandler`, 然后可以选择性地将它映射到一个 JDBC 类型。比如:

```
// ExampleTypeHandler.java
@MappedJdbcTypes(JdbcType.VARCHAR)
public class ExampleTypeHandler extends BaseTypeHandler<String> {

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, String parameter, JdbcType jdbcType) throws SQLException {
        ps.setString(i, parameter);
    }

    @Override
    public String getNullableResult(ResultSet rs, String columnName) throws SQLException {
        return rs.getString(columnName);
    }

    @Override
    public String getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
        return rs.getString(columnIndex);
    }

    @Override
    public String getNullableResult(CallableStatement cs, int columnIndex) throws SQLException {
        return cs.getString(columnIndex);
    }
}
```

**handler底层调用原生jdbc,
进行封装,不用管**

```
<!-- mybatis-config.xml -->
<typeHandlers>
  <typeHandler handler="org.mybatis.example.ExampleTypeHandler"/>
</typeHandlers>
```

2.1.5. objectFactory 对象工厂

2.1.6. plugins 插件(241集(可以不看))

2.1.7. environments 环境(242集(可以不看))

```
<!-- 6.environment :配置一个具体的环境;都需要一个事务管理器和一个数据源
transactionManager
后来数据源和事务管理都是Spring来做
-->
<!-- default="development":默认使用哪个环境 -->
<environments default="development">
  <!-- id="testEnv"是当前环境的唯一标识= -->
  <environment id="testEnv">
    <transactionManager type=""></transactionManager>
    <dataSource type="POOLED">

      <property name="driver" value="${driverclass}"/>
      <property name="url" value="${jdbcurl}"/>
      <property name="username" value="${username}"/>
      <property name="password" value="${password}"/>
    </dataSource>
  </environment>

  <environment id="development">
    <transactionManager type="JDBC"/>
    <!-- 配置连接池 -->
    <dataSource type="POOLED">
      <!-- ${}取出配置文件的值,这里不是EL表达式,凑巧一样罢了 -->
      <property name="driver" value="${driverclass}"/>
      <property name="url" value="${jdbcurl}"/>
      <property name="username" value="${username}"/>
      <property name="password" value="${password}"/>
    </dataSource>
  </environment>
</environments>
```

可以不用管

2.1.8. environment 环境变量

2.1.8.1. transactionManager 事务管理器

2.1.8.2. dataSource 数据源

2.1.9. databaseIdProvider 数据库厂商标识(243集)

```
<configuration>
The content of element type "configuration" must match
"(properties?,settings?,typeAliases?,typeHandlers?,objectFactory?,objectWrapperFactory?,plugins?,environments?,databaseIdProvider?,mappers?)".
```

注意标签的顺序

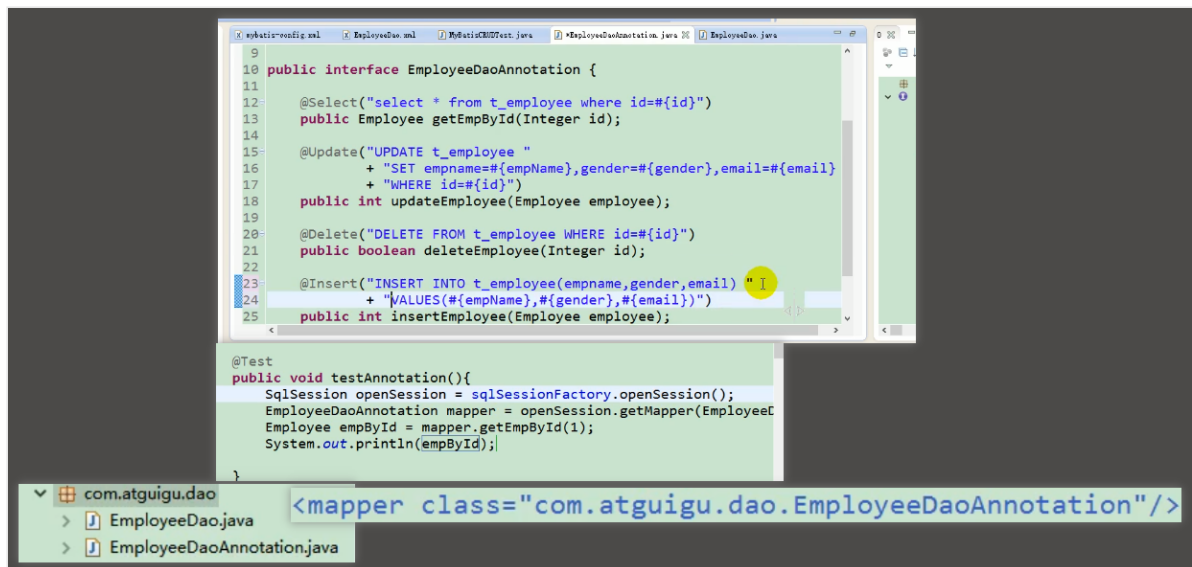
```
MyBatisCRUDTest.java Employee.java mybatis-config.xml EmployeeDao.xml

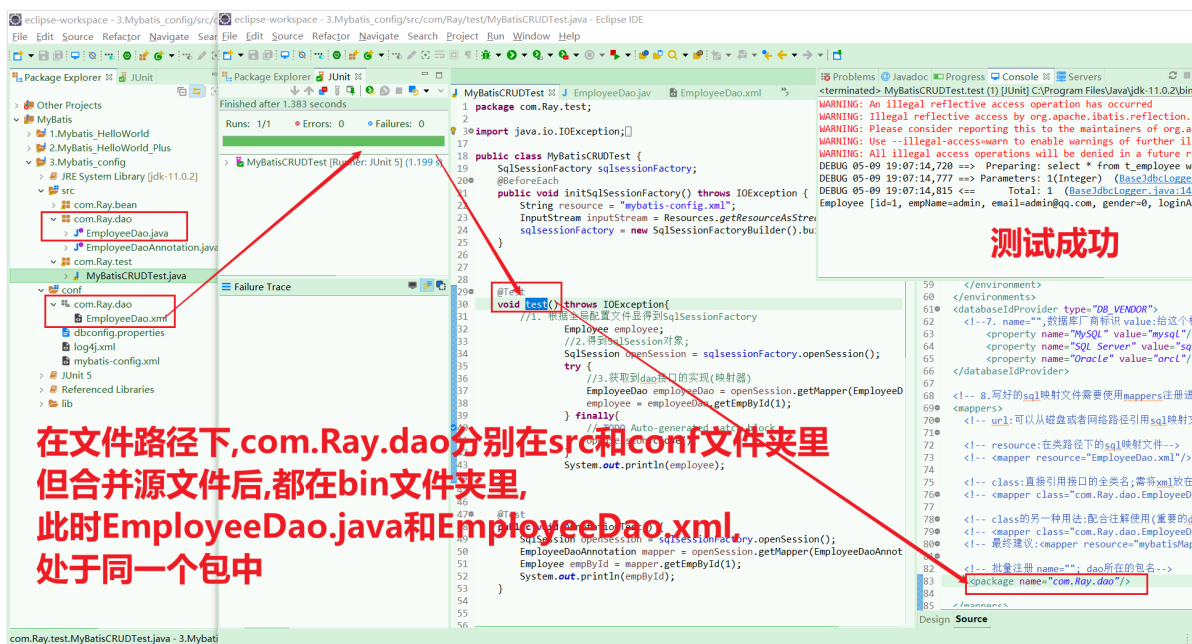
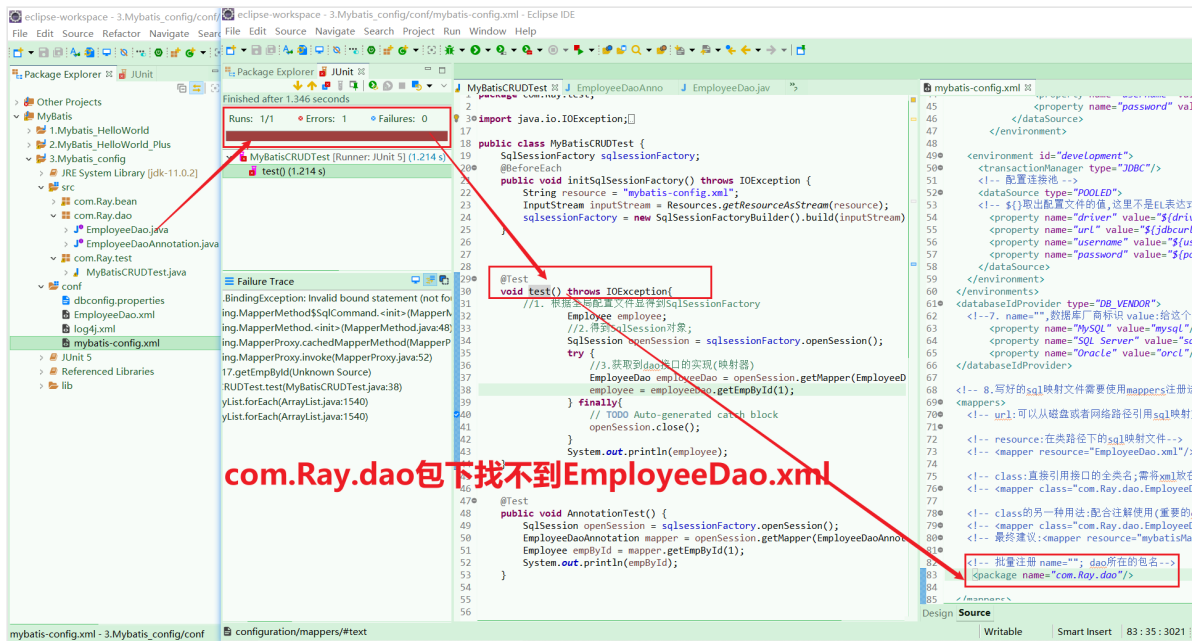
40 <dataSource type="POOLED">
41
42 <property name="driver" value="${driverclass}"/>
43 <property name="url" value="${jdbcurl}"/>
44 <property name="username" value="${username}"/>
45 <property name="password" value="${password}"/>
46 </dataSource>
47
48 <environment id="development">
49 <transactionManager type="JDBC"/>
50 <!-- 配置连接池 -->
51 <dataSource type="POOLED">
52 <!-- ${}取出配置文件的值,这里不是EL表达式,凑巧一样罢了 -->
53 <property name="driver" value="${driverclass}"/>
54 <property name="url" value="${jdbcurl}"/>
55 <property name="username" value="${username}"/>
56 <property name="password" value="${password}"/>
57 </dataSource>
58 </environment>
59 </environments>
60 <databaseIdProvider type="DB_VENDOR">
61 <!-- name="",数据库厂商标识 value:给这个标识起个好用的名字-->
62 <property name="MySQL" value="mysql"/>
63 <property name="SQL Server" value="sqlserver"/>
64 <property name="Oracle" value="oracle"/>
65 </databaseIdProvider>
66
67 <mappers>
68 <!-- 数据库厂商标识 -->
69 <!-- 数据库厂商标识 -->
70 <!-- 数据库厂商标识 -->
71 </mappers>
```

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3 PUBLIC "-//mybatis.org/DTD Mapper 3.0//EN"
4 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5
6
7 <mapper namespace="com.Ray.dao.EmployeeDao">
8 <!-- public Employee getEmpById(Integer id);返回值类型为全类名;参数类型不用写;-->
9 <select id="getEmpById" resultType="com.Ray.bean.Employee">
10 <!-- SQL语句不要写分号 -->
11 select * from t_employee where id = #{id}
12 </select>
13 <!-- public int updateEmployee(Employee employee);
14 增删改不用写返回值类型,增删改是返回影响多少行
15 Mybatis自动判断,如果是数字类型(int long,自动封装为数字)
16 如果是boolean,影响0行(封装为false,否则就是true)
17 #{属性名},从传入的参数对象中取出对应属性的值
18 -->
19 <update id="updateEmployee">
20 UPDATE t_employee
21 SET empname=#{empName},gender=#{gender},email=#{email}
22 WHERE id=#{id}
23 </update>
24 <select id="getEmpById" resultType="com.Ray.bean.Employee" databaseId="mysql">
25 select * from t_employee where id = #{id}
26 </select>
27 <select id="getEmpById" resultType="com.Ray.bean.Employee" databaseId="oracle">
28 select * from t_employee where id = #{id}
29 </select>
30
31 <!-- public int insertEmployee(Employee employee);-->
32
```

默认所有的

2.1.10. mappers 映射器(244集)

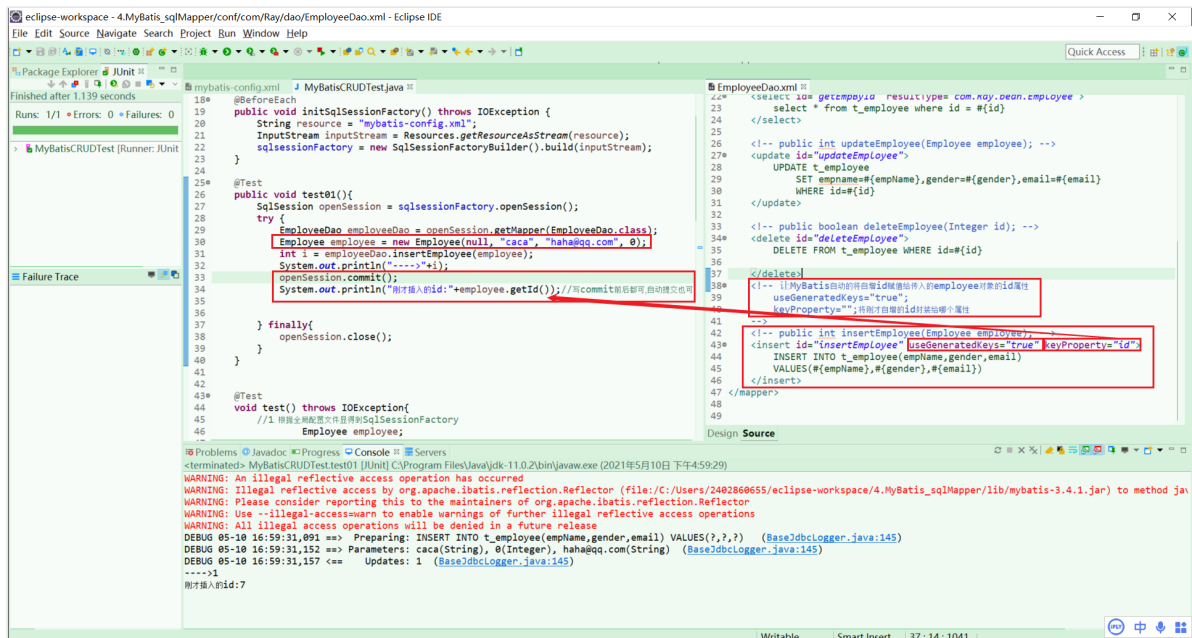




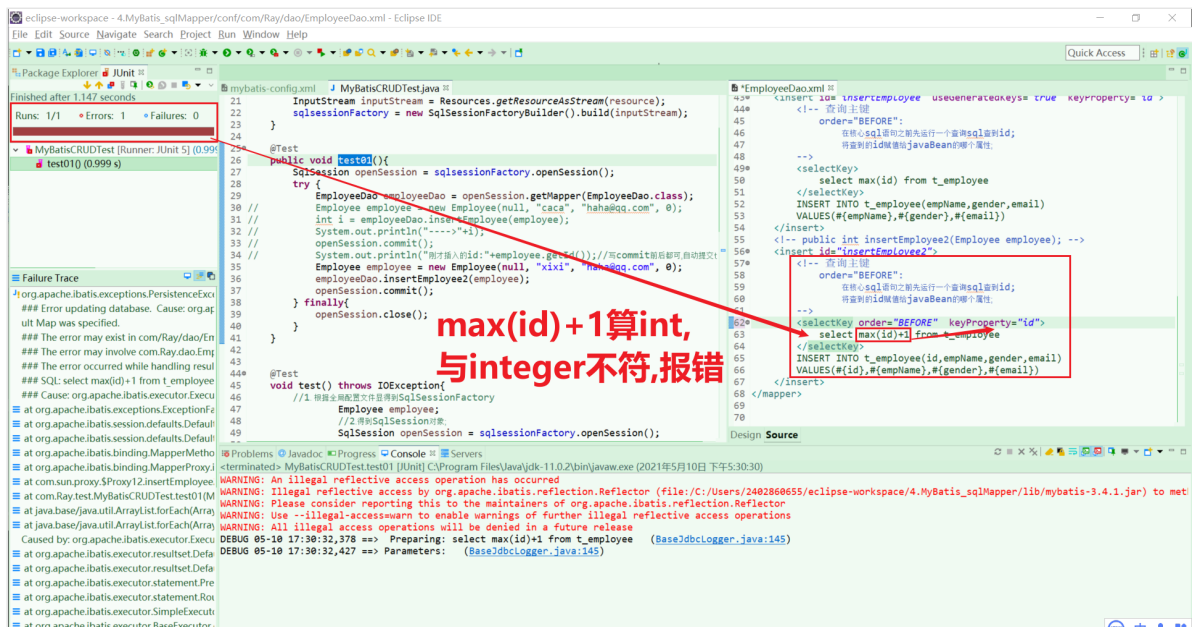
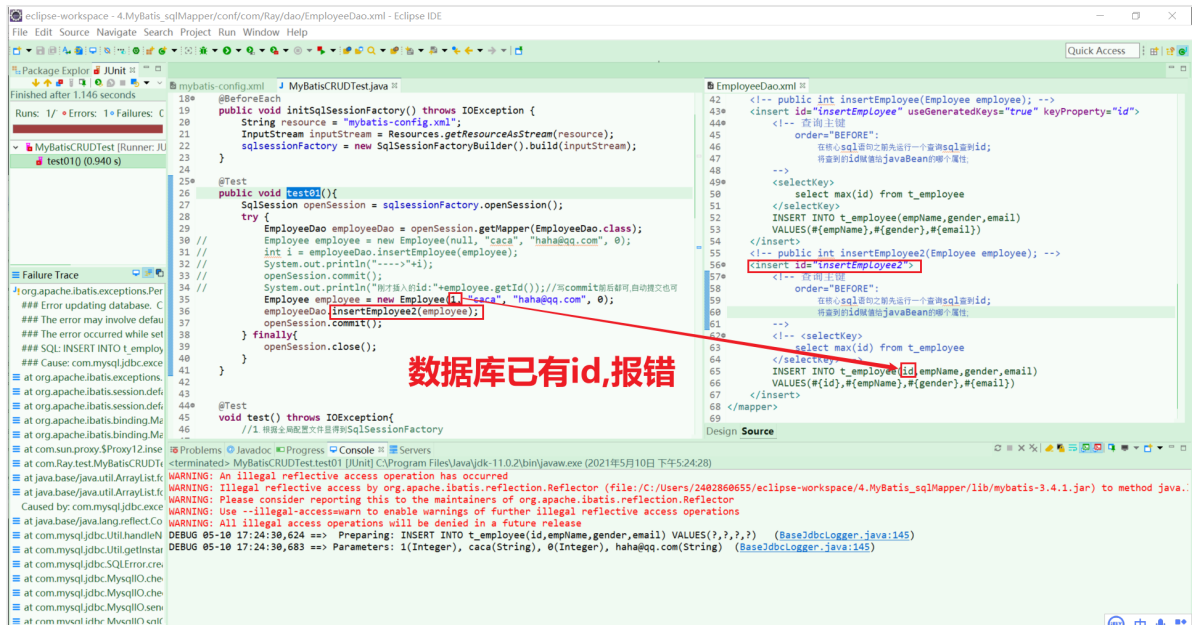
3. SQL配置文件(245-260集)

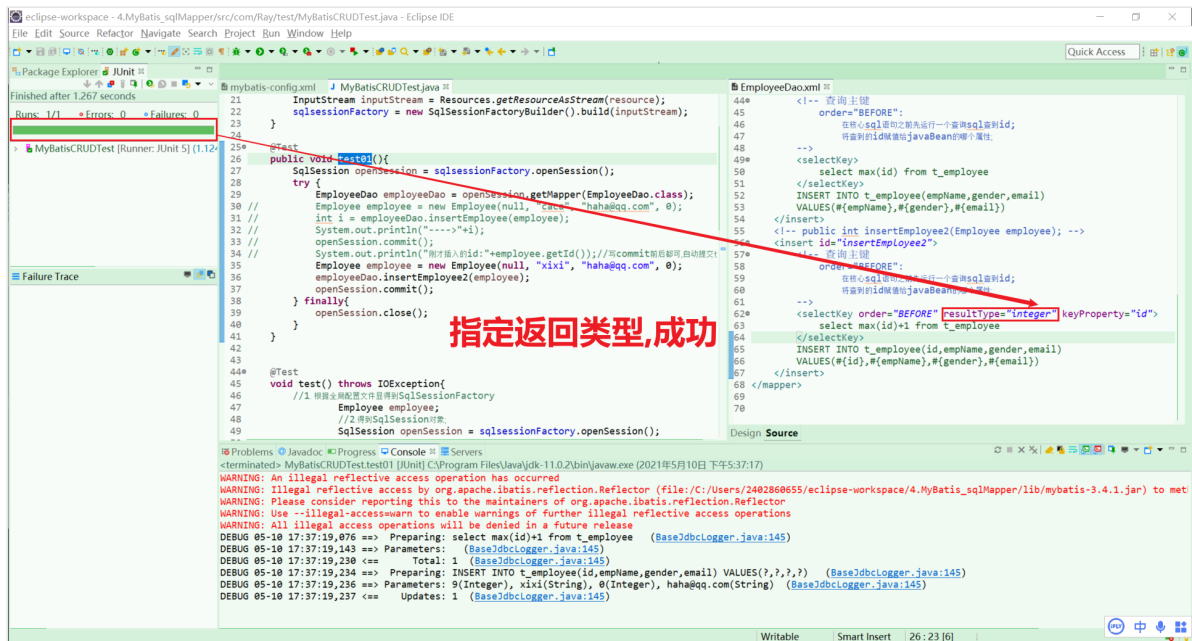
3.1. 详情见 [mybatis官方文档](#)(245集)

3.2. 获取自增主键的值(246集)

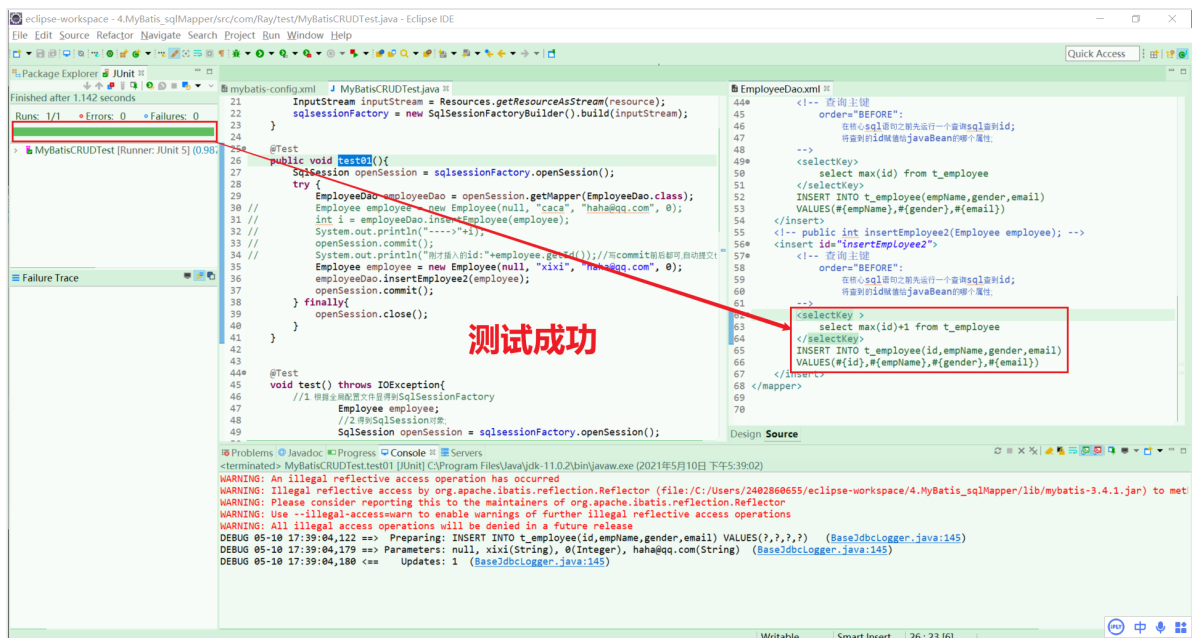


3.3. 获取非自增主键的值(247集)

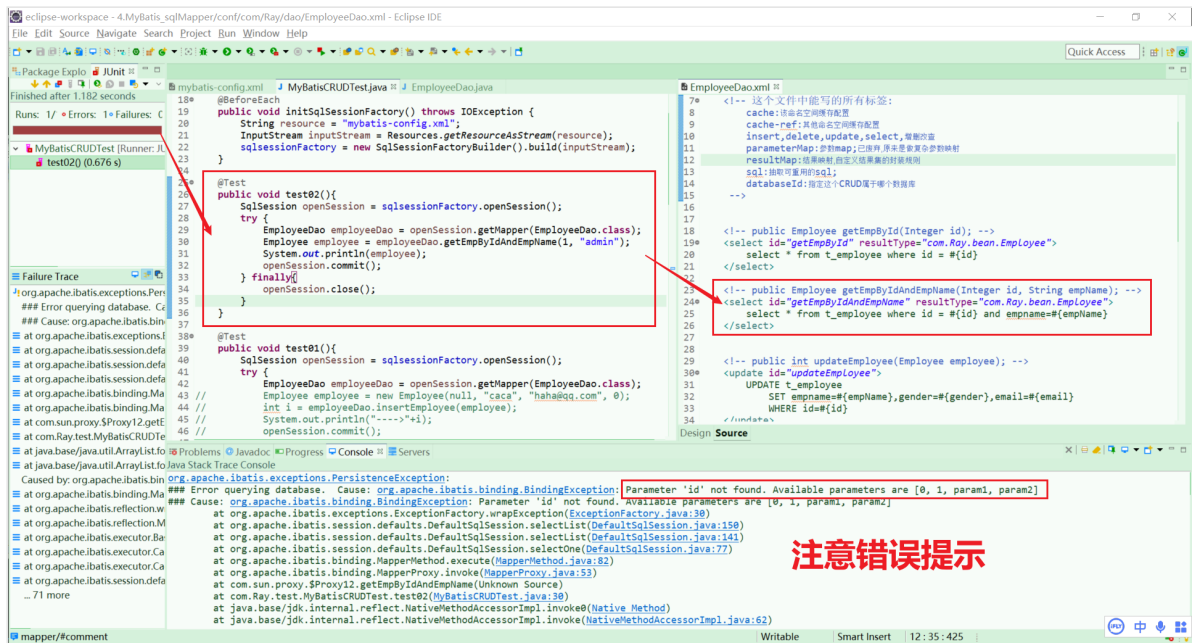




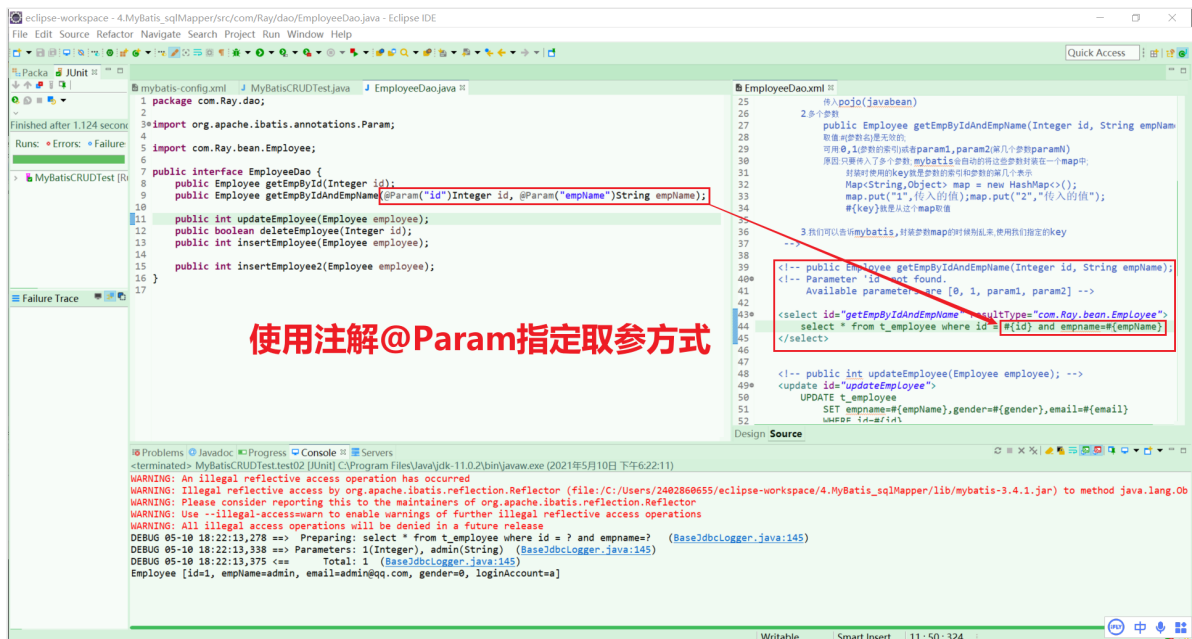
Tips:不写 order,resultType,keyProperty也能成功,但此时是自增的,而不是自己控制id+1,注意控制台 Parameters:null

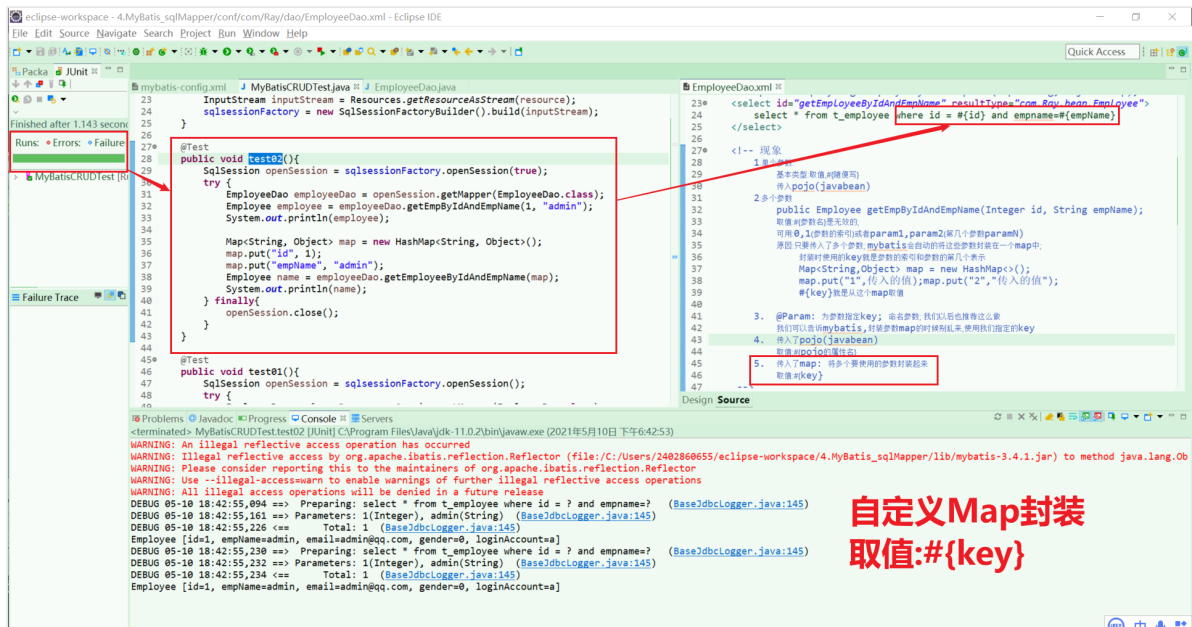


3.4. 参数的各种取值(248集)

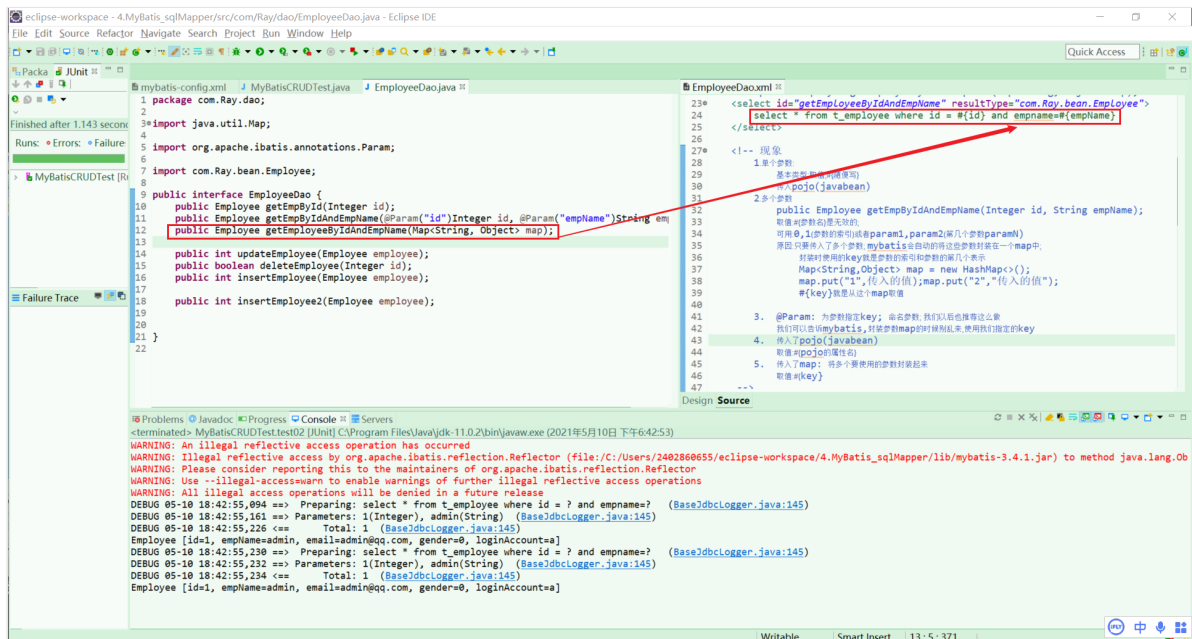


Parameter 'id' not found. Available parameters are [0, 1, param1, param2]





自定义Map封装 取值:#{key}



- <!-- 现象
1. 单个参数:
基本类型:取值,#{随便写}
传入pojo(javabeen)
2. 多个参数
public Employee getEmpByIdAndEmpName(Integer id, String empName);
取值:#{参数名}是无效的;
可用:0,1(参数的索引)或者param1,param2(第几个参数paramN)
原因:只要传入了多个参数;mybatis会自动的将这些参数封装在一个map中;
封装时使用的key就是参数的索引和参数的第几个表示
Map<String,Object> map = new HashMap<>();
map.put("1",传入的值);map.put("2",传入的值);
#{key}就是从这个map取值
3. @Param: 为参数指定key; 命名参数; 我们以后也推荐这么做
我们可以告诉mybatis,封装参数map的时候别乱来,使用我们指定的key
4. 传入了pojo(javabeen)
取值:#{pojo的属性名}
5. 传入了map: 将多个要使用的参数封装起来
取值:#{key}

```

21      扩展：多个参数：自动封装map；
22      method01(@Param("id")Integer id,String empName,Employee employee)
23      Integer id-> #{id}
24      String empName -> #{param2}
25      Employee employee(取出这个里面的email)->#{param3.email}
26
27      无论传入什么参数都要能正确的取出值；
28      #{key/属性名}
29      -->

```

3.5. SQL映射文件-oracle数据库情况下jdbcType需要被指定(249集)暂且跳过

3.6. SQL映射文件-#{ }和\${ }取值的区别(250集)

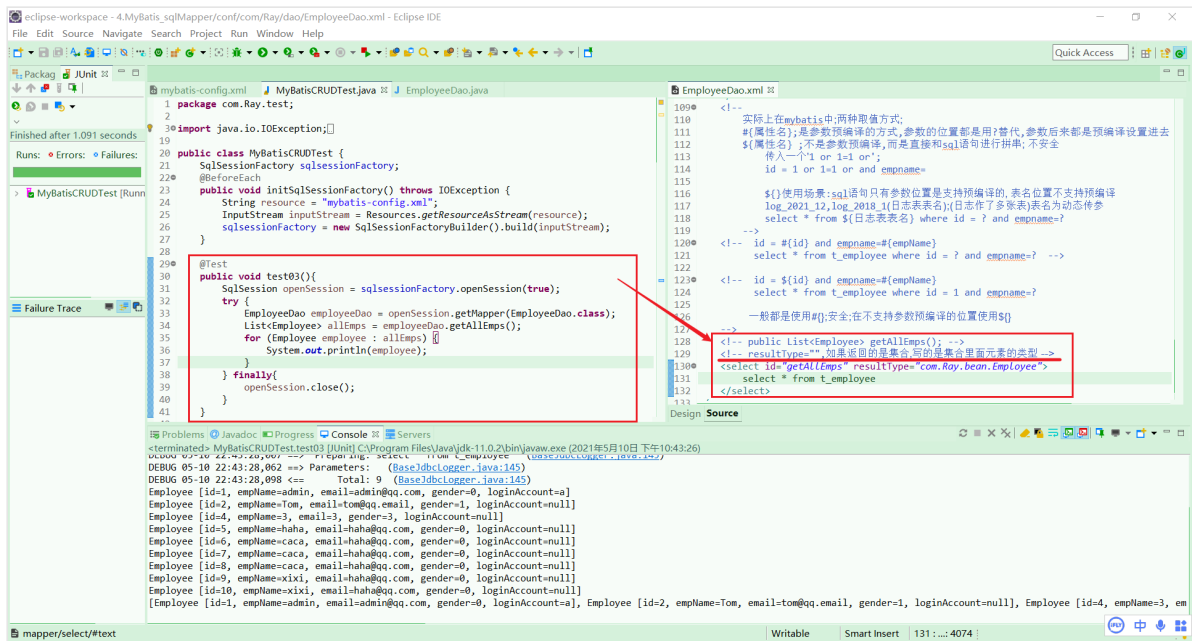
```

1  <!--
2      实际上在mybatis中；两种取值方式；
3      #{属性名}；是参数预编译的方式，参数的位置都是用?替代，参数后来都是预编译设置进去。安全，不会有sql注入问题
4      ${属性名}；不是参数预编译，而是直接和sql语句进行拼串； 不安全
5      传入一个'1 or 1=1 or'；
6      id = 1 or 1=1 or and empname=
7
8      ${ }使用场景：sql语句只有参数位置是支持预编译的，表名位置不支持预编译
9      log_2021_12,log_2018_1(日志表表名)；(日志作了多张表)表名为动态传参
10     select * from ${日志表表名} where id = ? and empname=?
11     -->
12     <!-- id = #{id} and empname=#{empName}
13         select * from t_employee where id = ? and empname=? -->
14
15     <!-- id = ${id} and empname=#{empName}
16         select * from t_employee where id = 1 and empname=?
17
18     一般都是使用#{ }；安全；在不支持参数预编译的位置使用${ }
19     -->

```

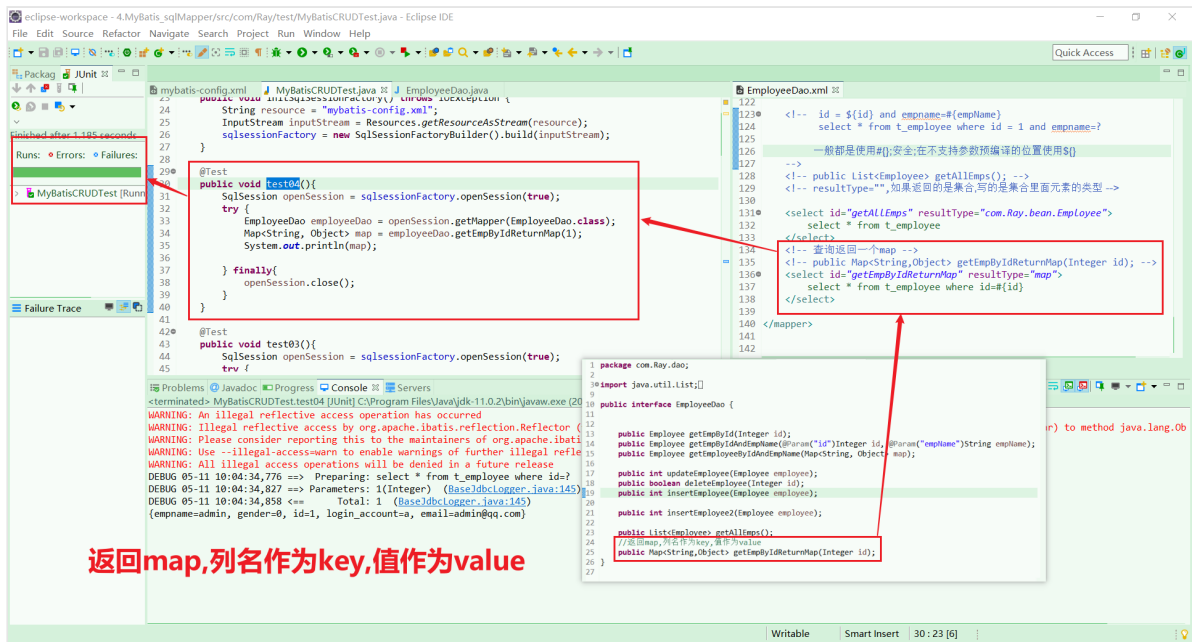
3.7. SQL映射文件-查询返回list(251集)

返回列表或集合,SQL映射文件resultType写元素的类型,而不是列表或集合本身



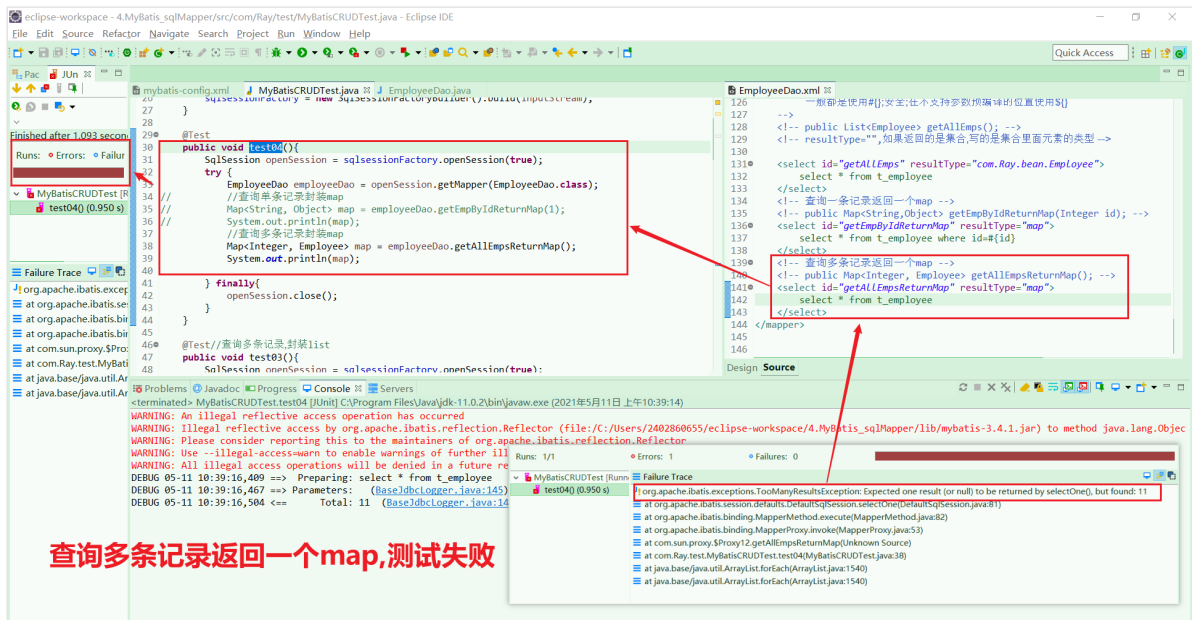
3.8. SQL映射文件-查询多个封装map(252集)

查询一条记录,返回一个map,列名作为key,值作为value

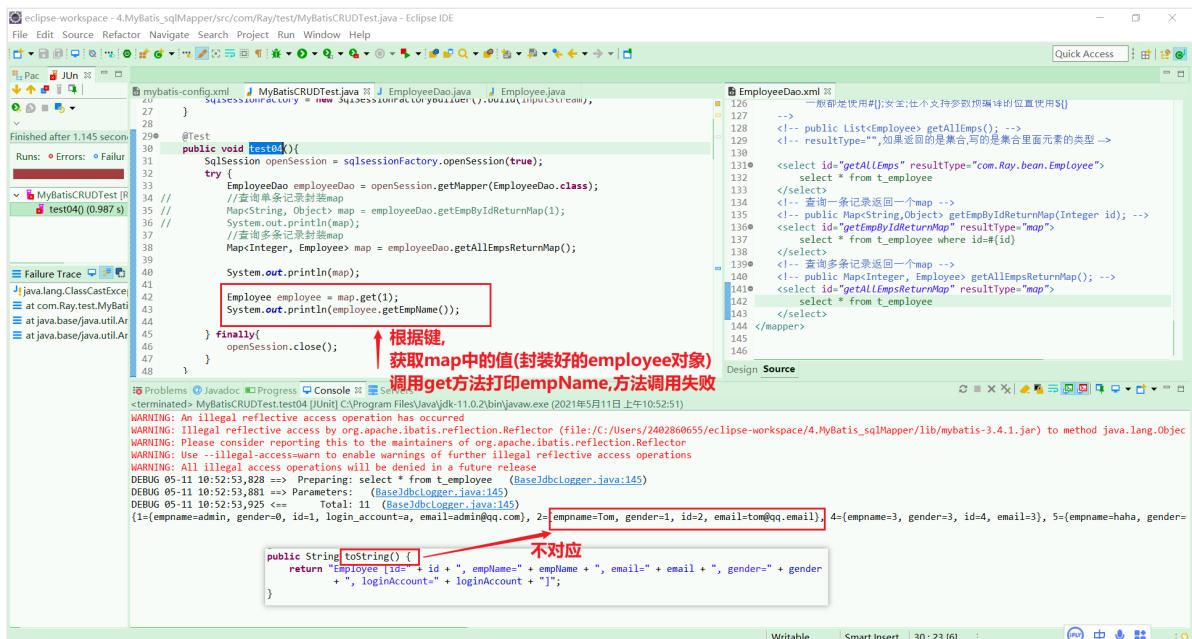
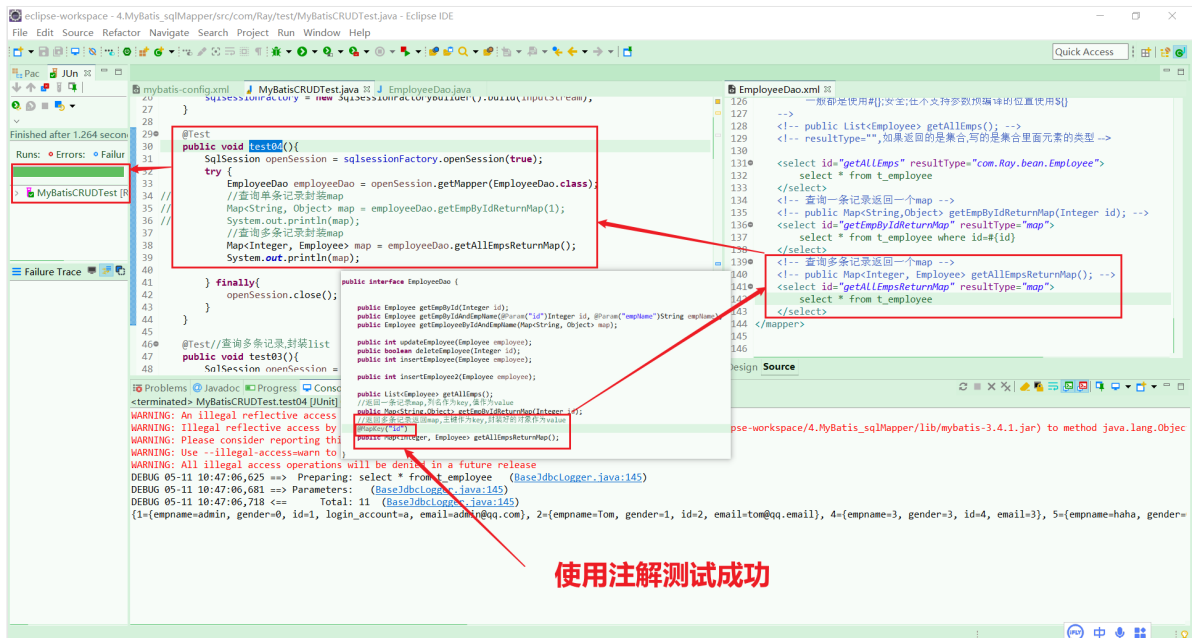


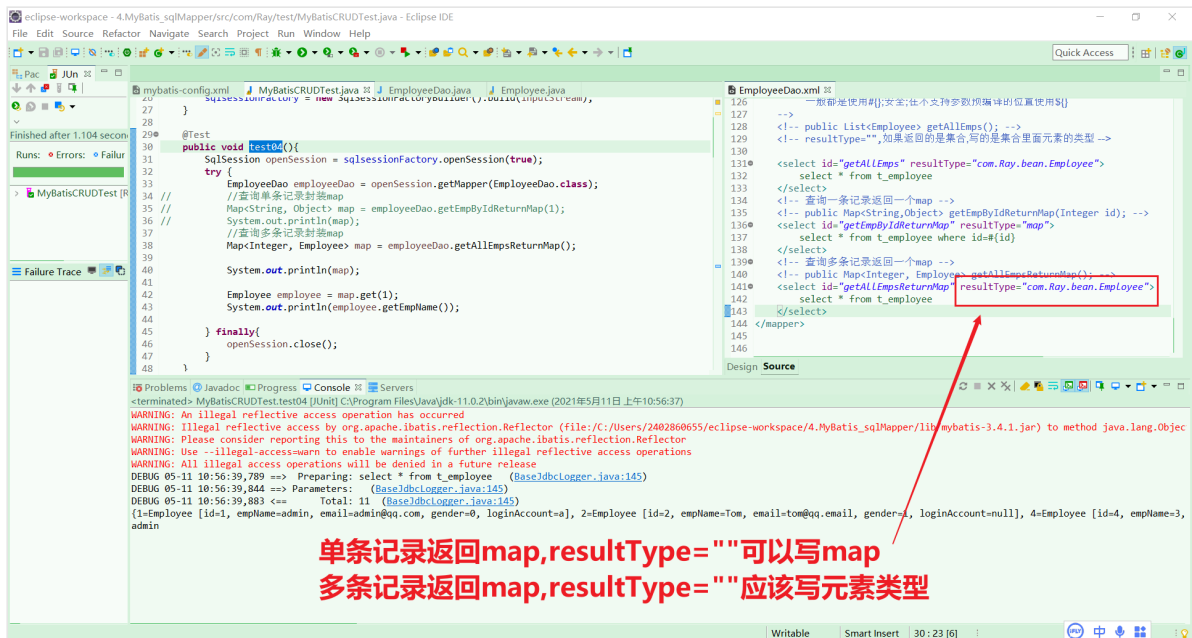
返回map,列名作为key,值作为value

查询多条记录,返回一个map,主键作为key,封装对象作为value,需使用注解@MapKey(id)



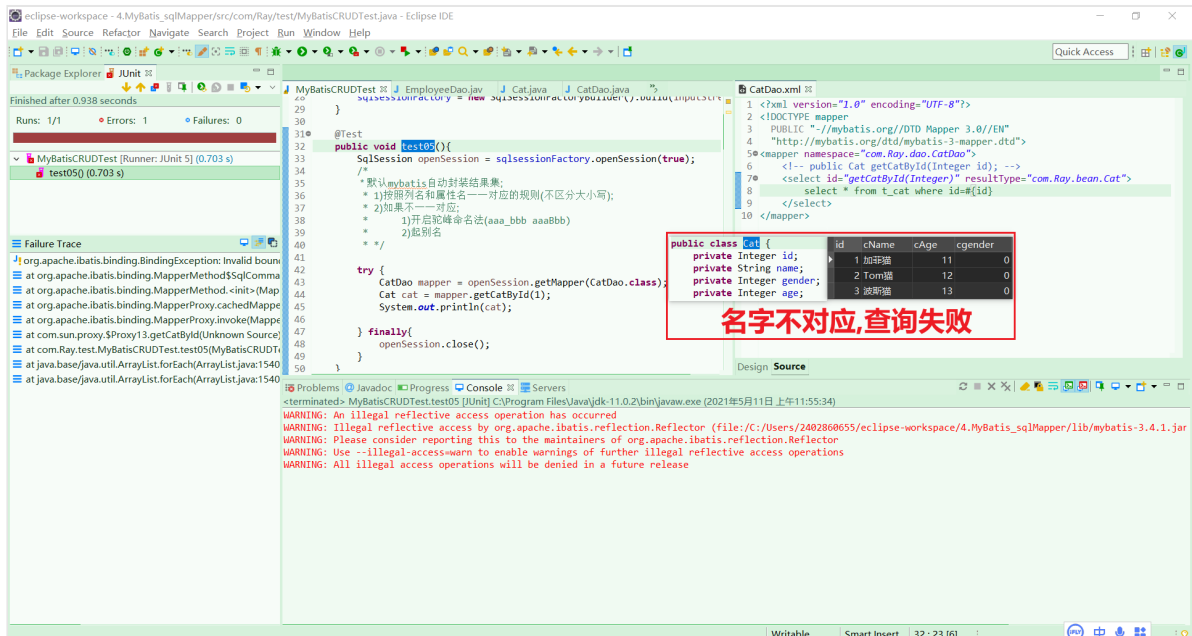
下图为使用注解测试成功,但是打印对象没有按照toString打印



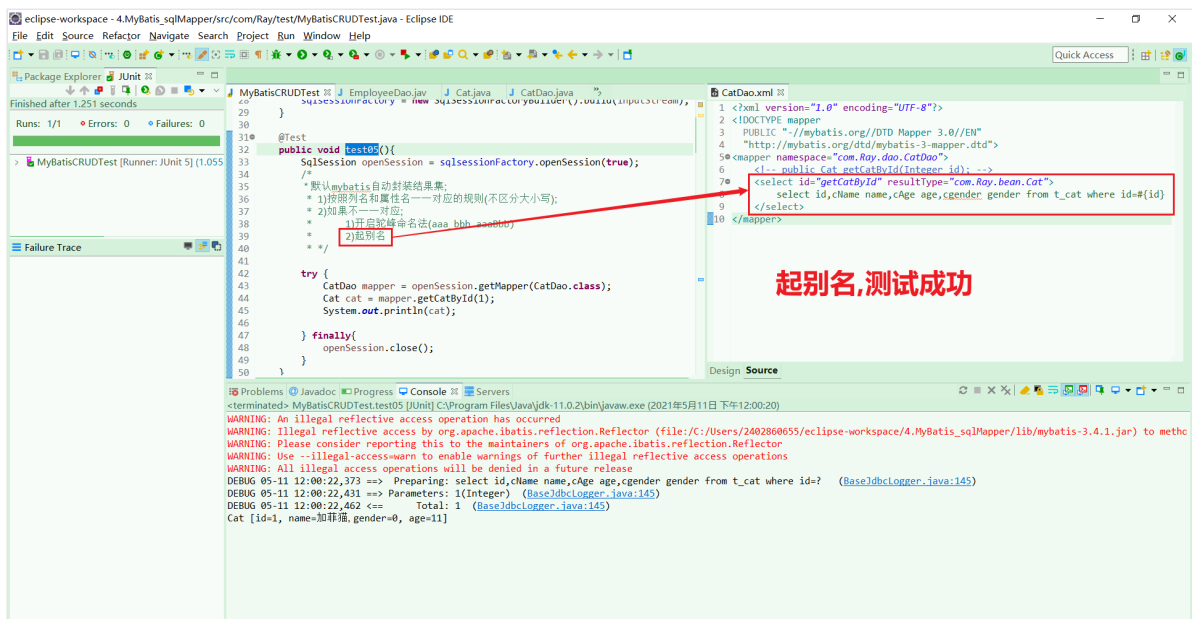


单条记录返回map,resultType=""可以写map
多条记录返回map,resultType=""应该写元素类型

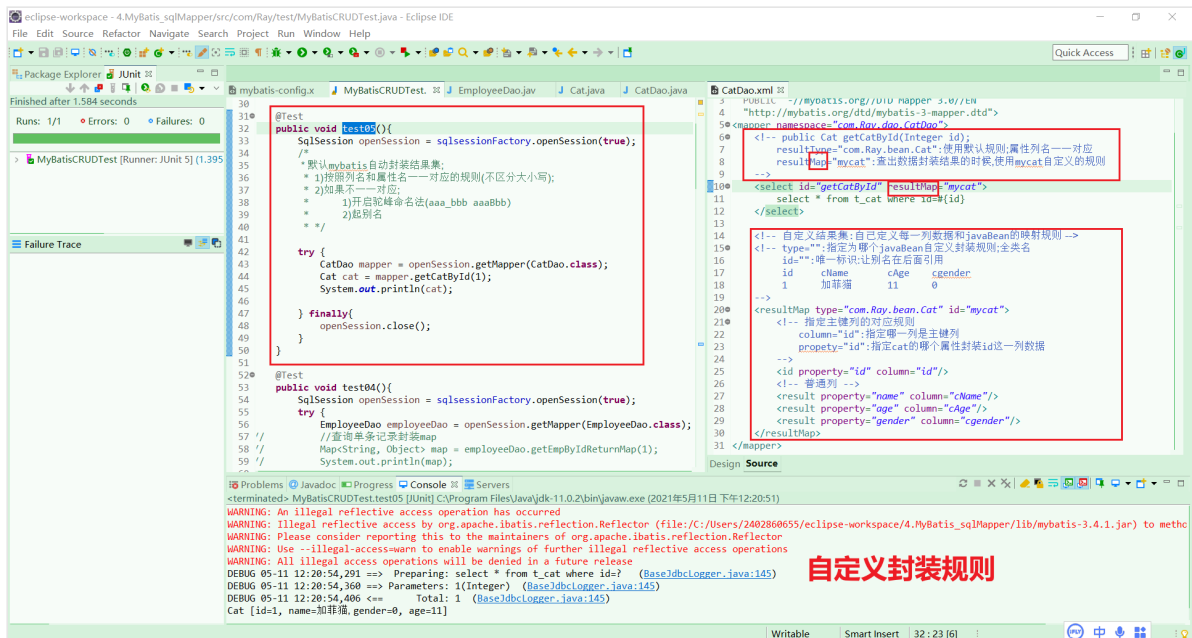
3.9. SQL映射文件-resultType自定义封装规则(253集)



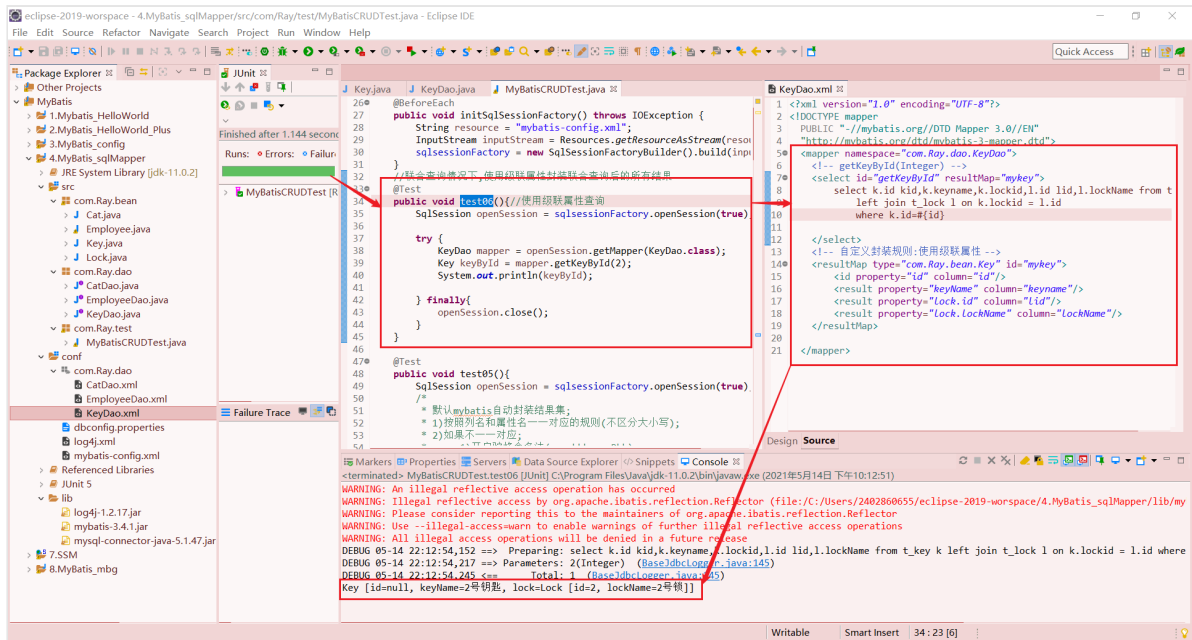
名字不对应,查询失败



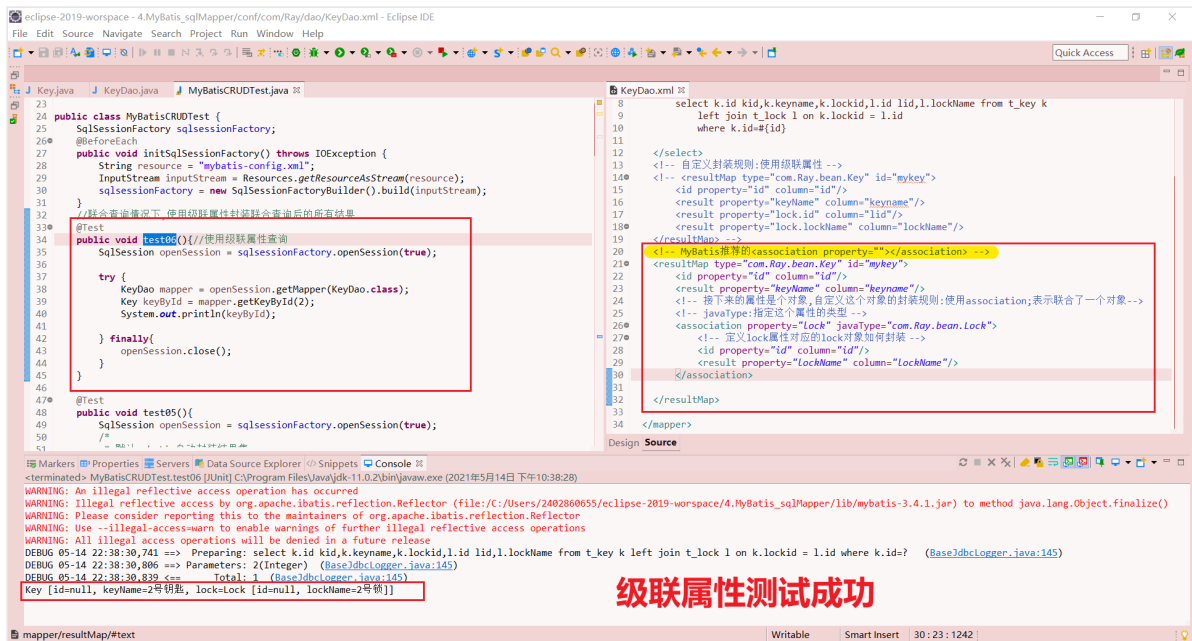
起别名,测试成功



3.10. SQL映射文件-联合查询-1、级联属性的方式封装查出的数据(254集)



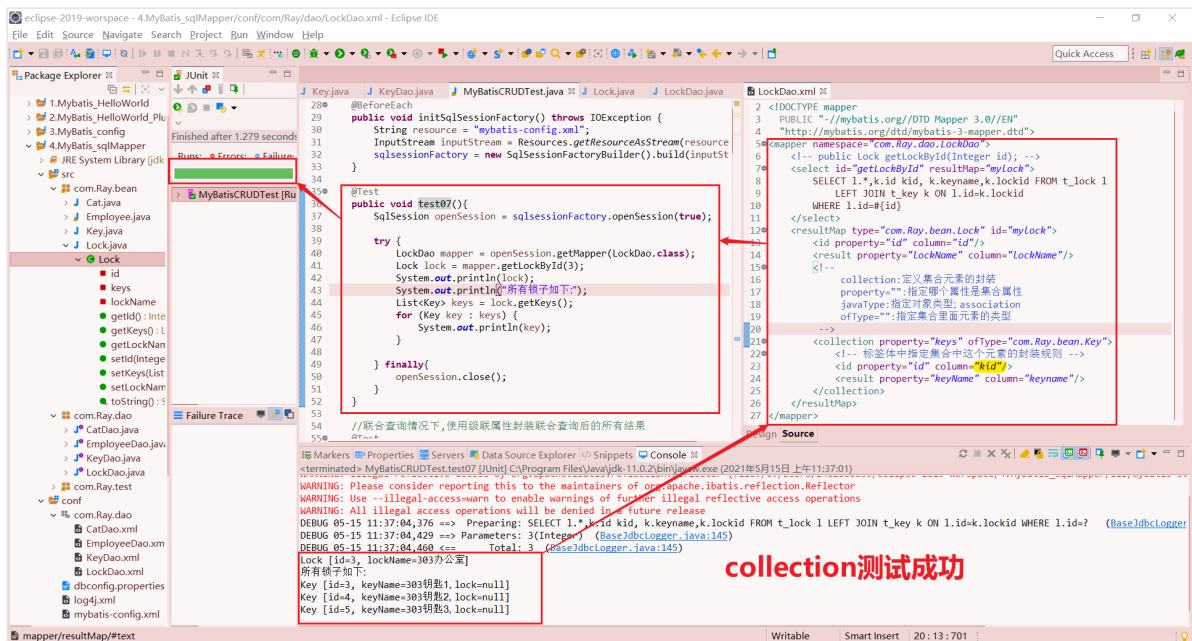
3.11. SQL映射文件-联合查询-2、使用association定义联合查询的对象的封装规则(255集)



3.12. SQL映射文件-联合查询-3、加谈-关联关系(256集)

1-1关联 1-n n-n关联
一个key开一把lock: 1-1
从lock来看key: 1-n
从key来看lock: n-1
n-n:学生和老师
一对n:外键一定放在n的一端
n-n:中间表存储对应关系
滚回去复习数据库设计范式!!!

3.13. SQL映射文件-联合查询-4、collection定义集合类型属性的封装规则(257集)



3.14. SQL映射文件-联合查询-5、使用select属性指定分步查询 (258集)

select属性指定分步查询测试成功

The screenshot shows the Eclipse IDE with the following components:

- KeyDao.java:** Contains the `test08()` method which calls `getByKeyIdSimple(1)` and `getLockByIdSimple(1)`.
- KeyDao.xml:** Contains two SQL mappings:
 - `getKeyBySimple`: `select * from t_key where id=#(id)`
 - `getLockBySimple`: `select * from t_lock where id=#(id)`
- Console:** Shows the execution of the test, with the following output:

```
DEBUG 05-15 13:28:55,980 ==> Preparing: select * from t_key where id=? (BaseJdbcLogger.java:145)
DEBUG 05-15 13:28:56,039 ==> Parameters: 1(Integer) (BaseJdbcLogger.java:145)
DEBUG 05-15 13:28:56,072 ==> Preparing: select * from t_lock where id=? (BaseJdbcLogger.java:145)
DEBUG 05-15 13:28:56,074 ==> Parameters: 1(Integer) (BaseJdbcLogger.java:145)
DEBUG 05-15 13:28:56,080 <==== Total: 1 (BaseJdbcLogger.java:145)
DEBUG 05-15 13:28:56,081 <==== Total: 1 (BaseJdbcLogger.java:145)
Key [id=1, keyName=1号钥匙, lockLock [id=1, lockName=1号锁]]
```

3.15. SQL映射文件-联合查询-6、按需加载和延迟加载(259集)

按需加载

只查KeyName

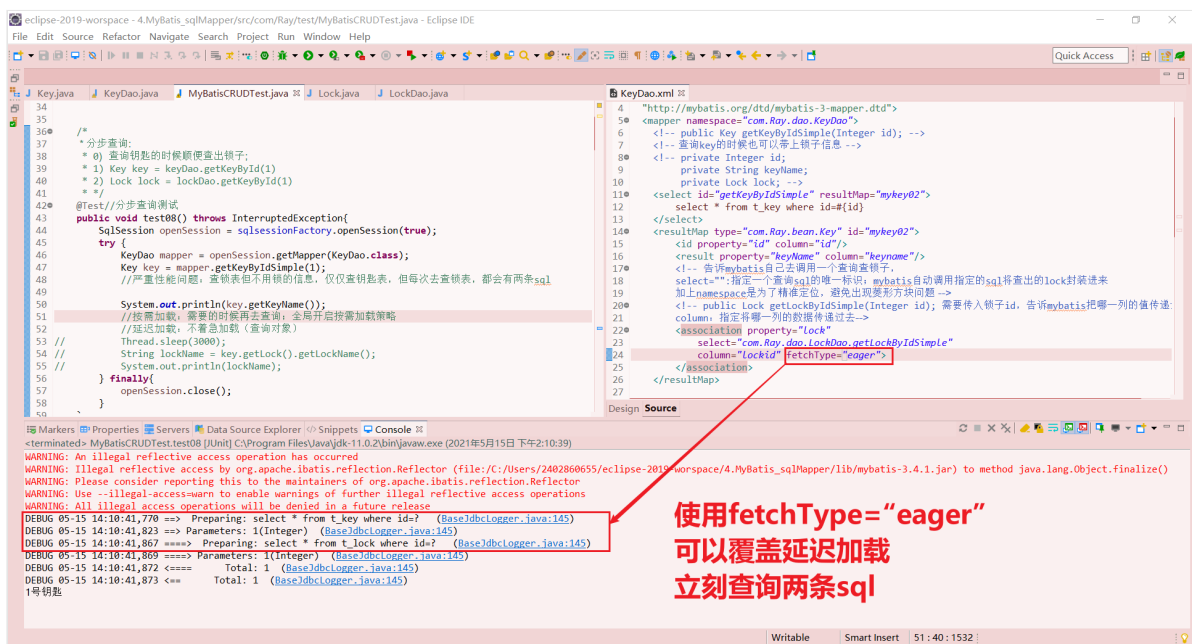
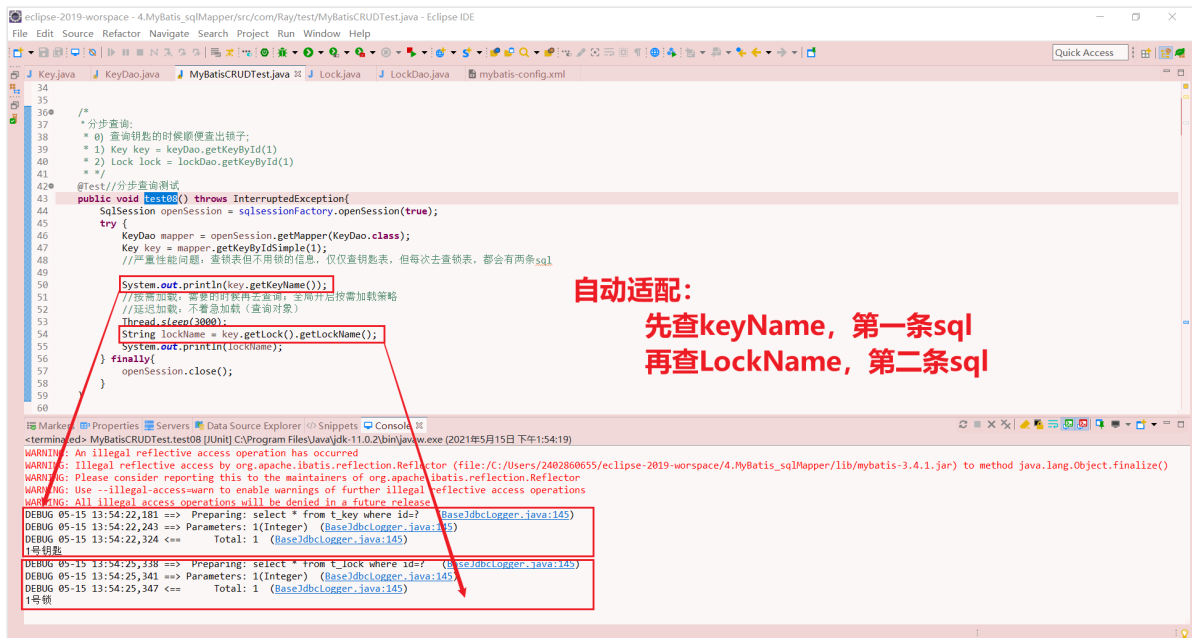
此处只有一条sql

The screenshot shows the Eclipse IDE with the following components:

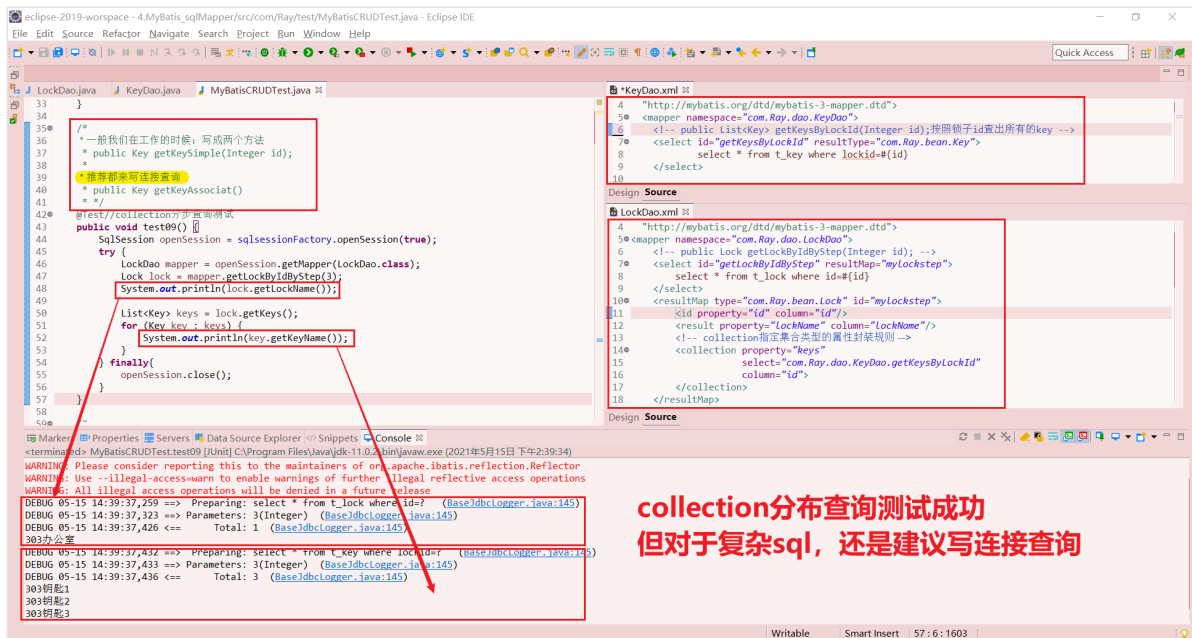
- mybatis-config.xml:** Contains the configuration for lazy loading and aggressive lazy loading:

```
<settings>
  <setting name="lazyLoadingEnabled" value="true"/>
  <setting name="aggressiveLazyLoading" value="false"/>
</settings>
```
- KeyDao.java:** Contains the `test08()` method which calls `getByKeyIdSimple(1)` and `getLockByIdSimple(1)`.
- Console:** Shows the execution of the test, with the following output:

```
DEBUG 05-15 13:48:15,238 ==> Preparing: select * from t_key where id=? (BaseJdbcLogger.java:145)
DEBUG 05-15 13:48:15,353 ==> Parameters: 1(Integer) (BaseJdbcLogger.java:145)
DEBUG 05-15 13:48:15,391 <==== Total: 1 (BaseJdbcLogger.java:145)
1号钥匙
```

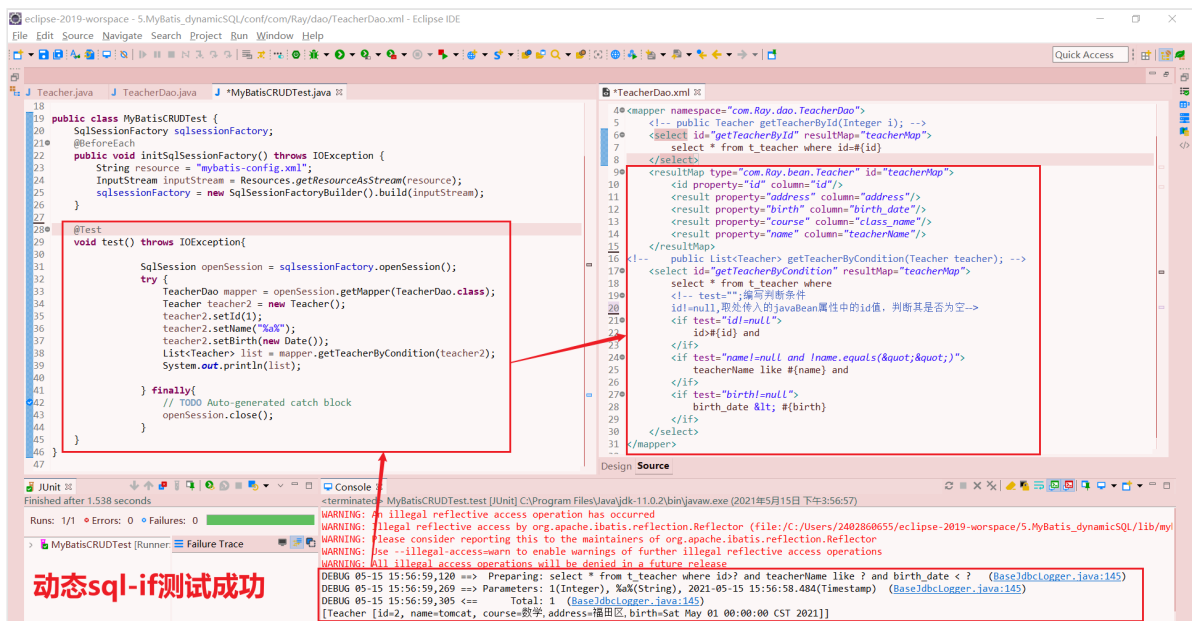


3.16. SQL映射文件-联合查询-7、collection分步查询延迟加载测试(260集)

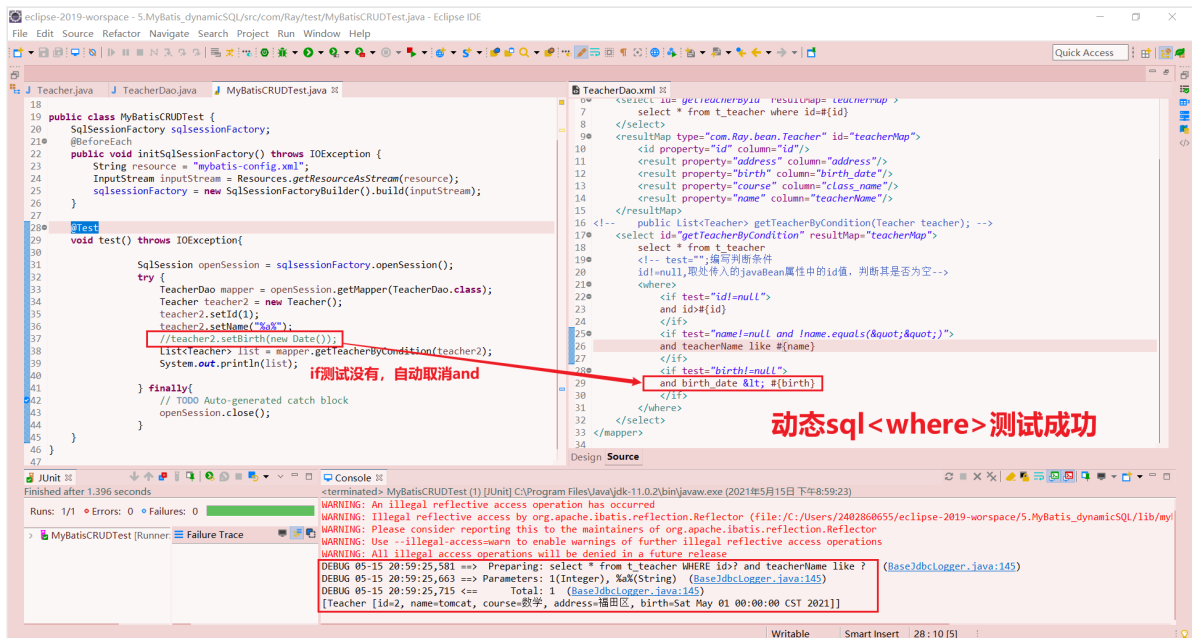


4. 动态Sql(261-267集)

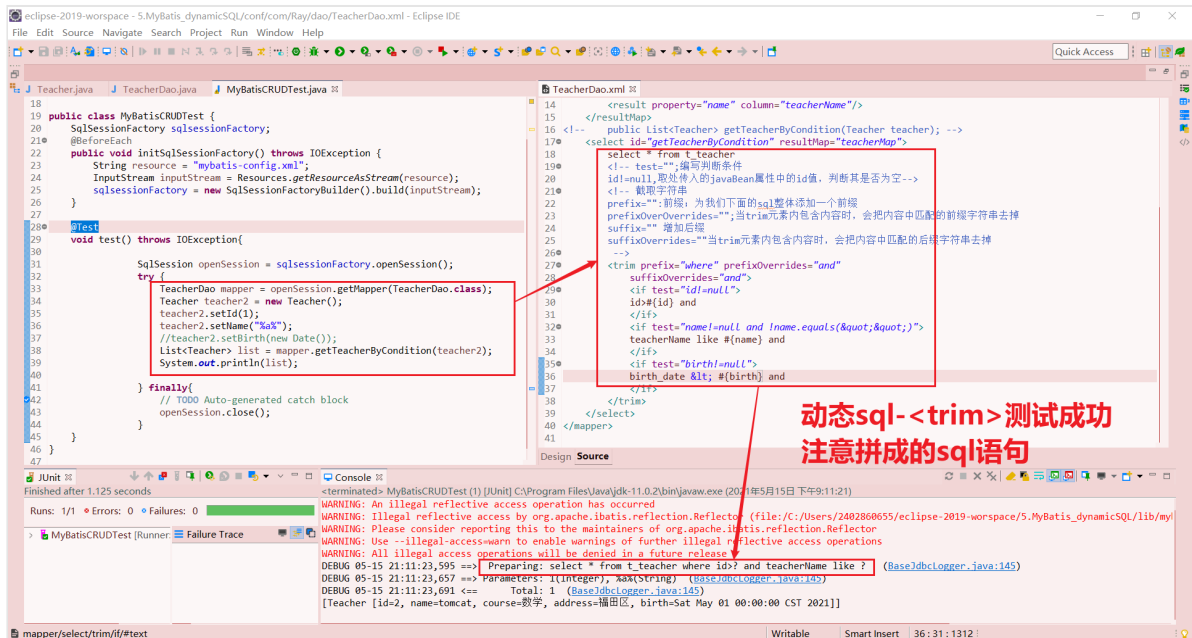
4.1. 动态sql-if标签(261集)



4.2. 动态sql-where标签(262集)



4.3. 动态sql-trim标签(263集)



4.4. 动态sql-foreach遍历元素(264集)

- 1 <!-- <foreach>帮我们遍历集合的； collection=""；指定要遍历的集合的key list参数的类型，
- 2 填list(collection="list")能取出来，用参数名ids(collection="ids")不能，
- 3 但在用@Param指定参数名后就可以了-->
- 4 <!--
- 5 index="" 索引
- 6 如果遍历的是一个list：
- 7 index:指定的变量保存了当前索引
- 8 item:保存当前遍历的元素的值
- 9 如果遍历的是一个map：
- 10 index:指定的变量就是保存了当前遍历的元素的key
- 11 item:就是保存当前遍历的元素的值
- 12 item="" 变量名：每次遍历出的元素起一个变量名方便引用
- 13 open="" 以什么开始
- 14 close="" 以什么结束
- 15 separator="" 每个遍历的元素的分隔符

动态sql-<foreach>测试成功

```

//test
void test() throws IOException{
    SqlSession openSession = sqlSession.openSession();
    try {
        TeacherDao mapper = openSession.getMapper(TeacherDao.class);
        Teacher teacher2 = new Teacher();
        teacher2.setId(1);
        teacher2.setName("张三");
        //teacher2.setBirth(new Date());
        List<Teacher> list = mapper.getTeacherByIdIn(Arrays.asList(1,2,3,4,5));
        for (Teacher teacher : list) {
            System.out.println(teacher);
        }
    } finally{
        // TODO Auto-generated catch block
        openSession.close();
    }
}

```

```

<!--
    public List<Teacher> getTeacherByIdIn(List<Integer> ids); -->
    <select id="getTeacherByIdIn" resultMap="teacherMap">
    <!-- select * from t_teacher where id IN (1,2,3,4,5) -->
    select * from t_teacher where id IN
    <!-- <foreach>帮我们遍历集合的；collection=""，指定要遍历的集合的key list参数的类型，
    填list(collection="list")能取出来，用参数名ids(collection="ids")不能，
    但在使用@Param指定参数名后就可以了 -->
    <!--
    index="" 索引
    如果遍历的是一个list:
    index:指定的变量保存了当前索引
    item:保存当前遍历的元素值
    如果遍历的是一个map:
    index:指定的变量就是保存了当前遍历的元素key
    item:就是保存当前遍历的元素值
    item="" 变量名：每次遍历出的元素起一个变量名方便引用
    open="" 以什么开始
    close="" 以什么结束
    separator="" 每个遍历的元素的分隔符
    -->
    <foreach collection="ids" item="id_item"
    separator="," open="(" close=")">
    #id_item
    </foreach>
    </select>
    </mapper>

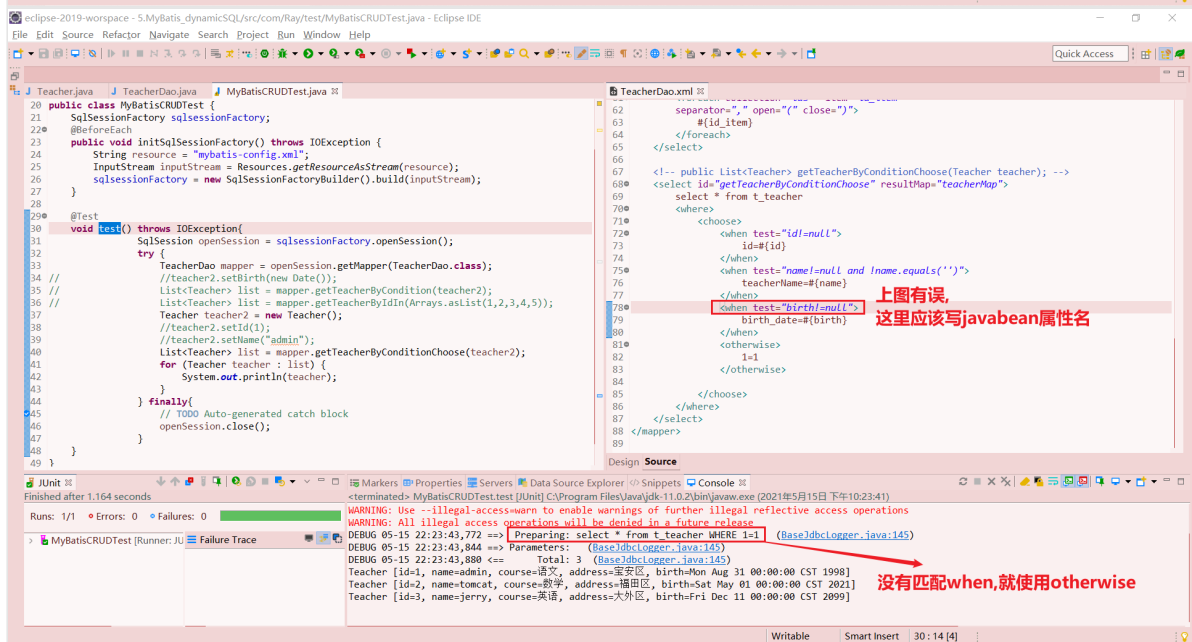
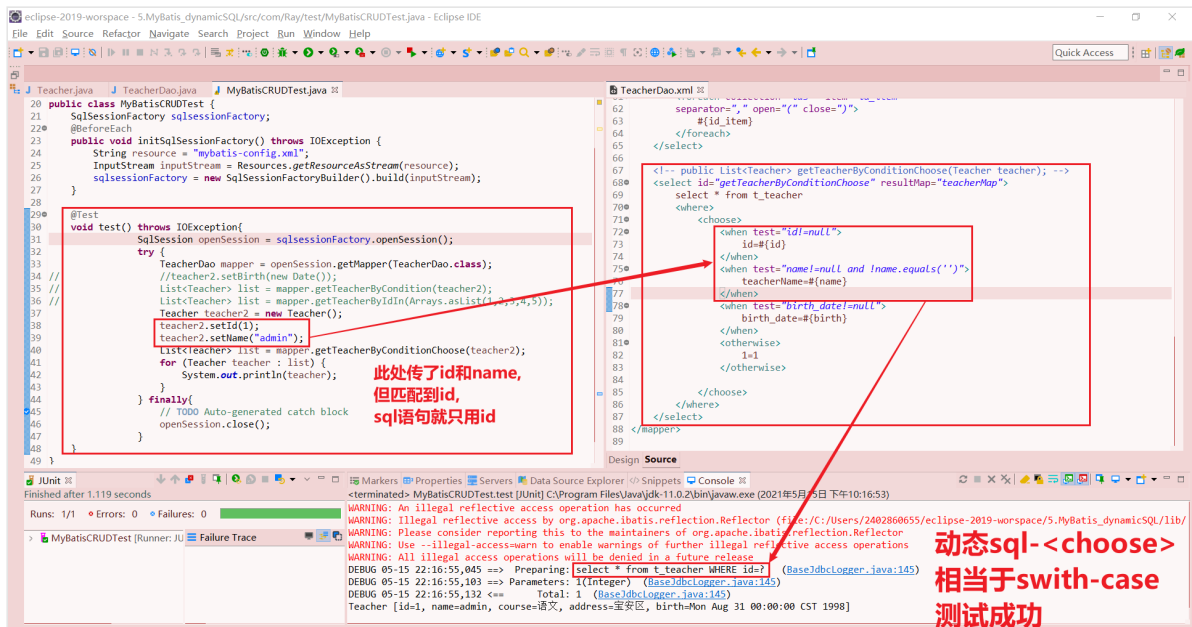
```

```

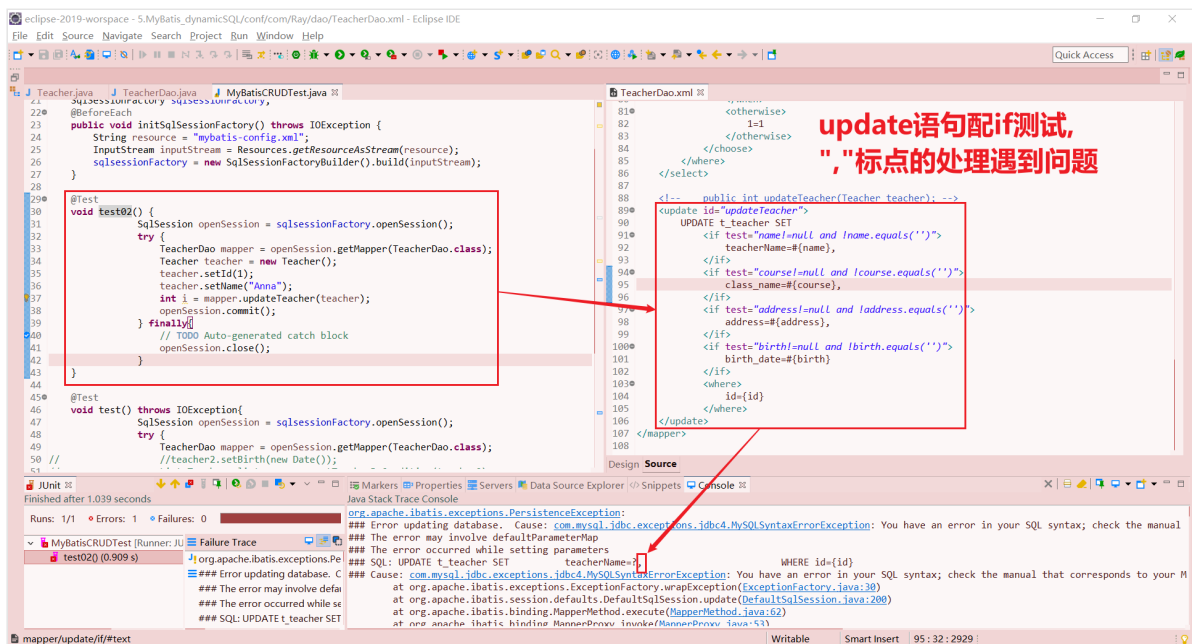
WARNING: Please consider reporting this to the maintainers of org.apache.ibatis.reflection.Reflector
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
DEBUG 05-15 21:42:14,260 ==> Preparing: select * from t_teacher where id IN ( ?, ?, ?, ?, ? ) (BaseJdbcLogger.java:145)
DEBUG 05-15 21:42:14,316 ==> Parameters: 1(Integer), 2(Integer), 3(Integer), 4(Integer), 5(Integer) (BaseJdbcLogger.java:145)
DEBUG 05-15 21:42:14,353 <== Total: 3 (BaseJdbcLogger.java:145)
Teacher [id=1, name=admin, course=语文, address=宝来区, birth=Mon Aug 31 00:00:00 CST 1990]
Teacher [id=2, name=tomcat, course=数学, address=福田区, birth=Sat May 01 00:00:00 CST 2021]
Teacher [id=3, name=jerry, course=英语, address=大外区, birth=Fri Dec 11 00:00:00 CST 2099]

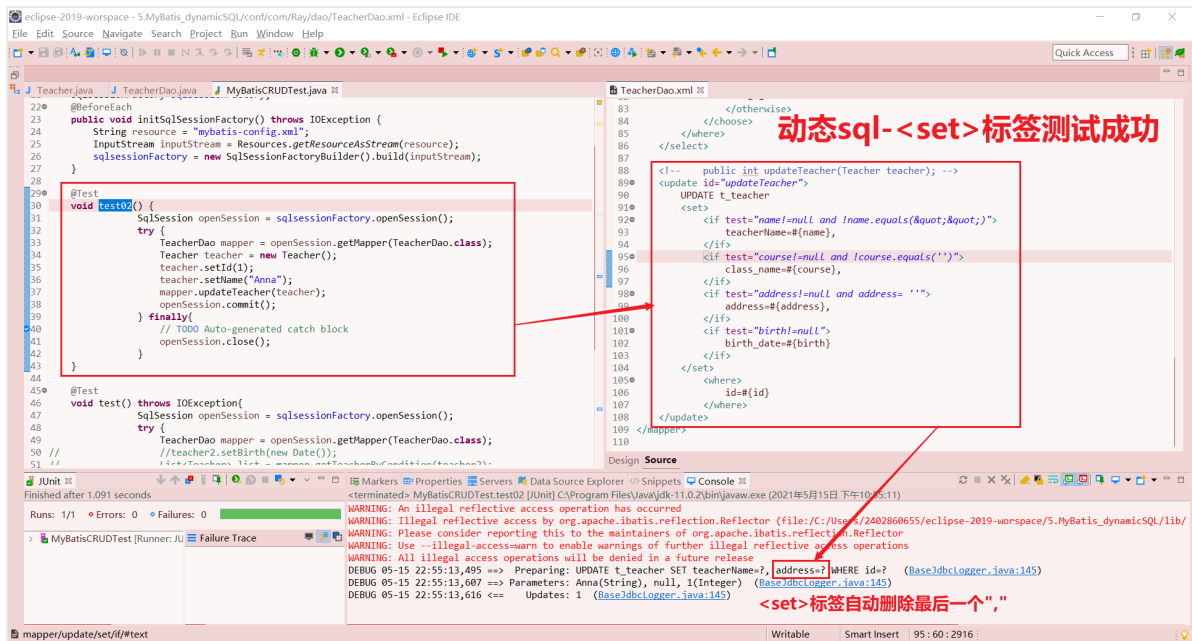
```

4.5. 动态sql-choose分支选择(265集)



4.6. 动态sql-使用if结合set完成mybatis动态更新(266集)





4.7. 动态sql-补充-OGNL和其他两个可用的参数(267集)

OGNL表达式；对象导航图

Person

- ===lastName
- ===email
- ===Address
- ===city
- ===province
- ===Street
- ===adminName
- ===info
- ===perCount

方法、静态方法、构造器。xxx

在mybatis中，传入的参数可以用来做判断；

额外还有两个东西；

- _parameter:** 代表传入来的参数；
 - 1)、传入了单个参数：_parameter就代表这个参数
 - 2)、传入了多个参数：_parameter就代表多个参数集合起来的map
- _databaseId:** 代表当前环境

如果

OGNL表达式单引号双引号不需要转义

```
-->
<if test="_databaseId=='mysql'">
    select * from sss
</if>
<if test="_databaseId=='oracle'">
    select * from oracle
</if>
```

写了这个就不用在SQL语句中用databaseId属性了

4.8. 动态sql-sql标签bind+抽取可重用的sql, 使用include包含进来(268集)

标签:可以使用ONGL表达式创建一个变量并将其绑定到上下文中

```
1 <if test="username" != null and userName != ''>
2     and user_name like concat('%',{userName},'%')
3 </if>
4 <!-- 如果使用concat函数连接字符串,在MySQL中,这个函数支持多个参数,但在Oracle中只支持两个
   参数.由于不同数据库之间的语法差异,如果更换数据库,有些SQL语句可能就需要重写.针对这种情况,可
   以使用bind标签来避免由于更换数据库带来的麻烦.将上面的方法改为bind方式后,代码如下 -->
5 <if test="username" != null and userName != ''>
6     <bind name="userNameLike" value="'%' + userName + '%'" />
7     and user_name like #{userNameLike}
8 </if>
9 <!-- <bind>标签的两个属性都是 -->
```

标签:抽取可重用的sql语句

```
<!--抽取可重用的sql语句 -->
<sql id="selectSql">select * from t_teacher</sql>
<!--public Teacher getTeacherById(Integer id); -->
<select id="getTeacherById" resultMap="teacherMap">
    <include refid="selectSql"></include>
    where id=#{id}
</select>
```

5. 缓存(269-276集)

5.1. 缓存-缓存简介(269集)

缓存: 暂时的存储一些数据; 加快系统的查询速度

CPU: 主频: 4-2.7GHZ

内存: 4G-8G 1333MHZ 2166MHZ

CPU: 一级缓存(4MB) 二级缓存(16MB)

数据库交互:

```
Employee employee = employDao.getEmpById(1);
```

MyBatis缓存机制: Map; 能保存查询出的一些数据

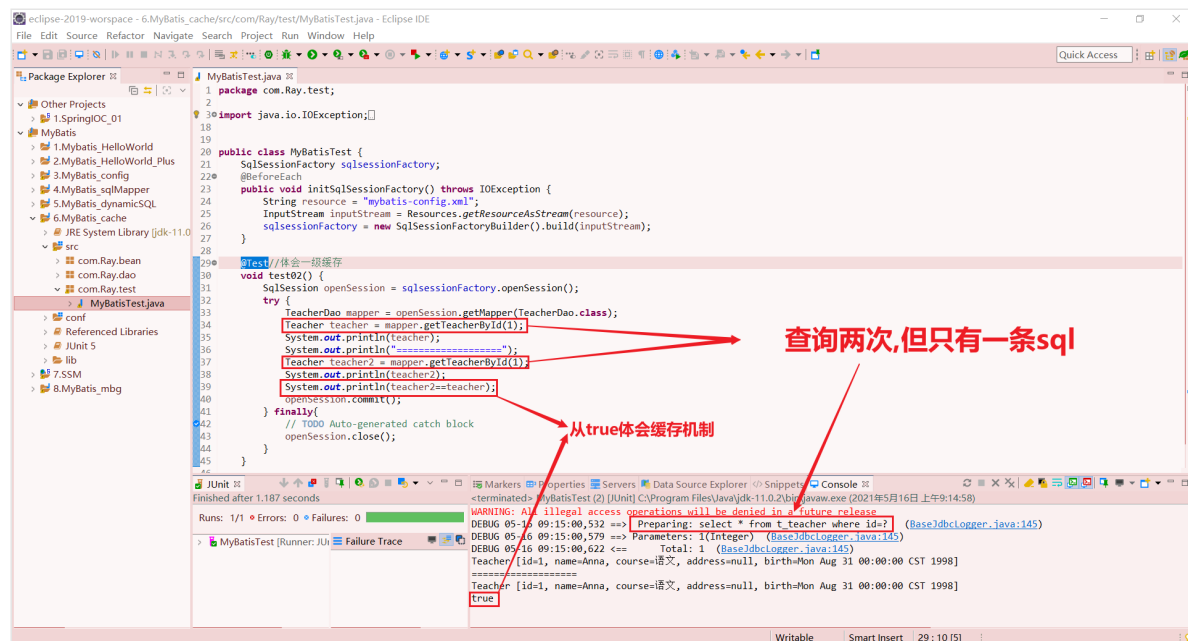
一级缓存:线程级别的缓存; 本地缓存; SqlSession级别的缓存

二级缓存:全局范围的缓存; 除过当前线程; SqlSession能用外其他也可以使用

5.2. 缓存-一级缓存体会(270集)

一级缓存: MyBatis: SqlSession级别的缓存; 默认存在;

机制: 只要之前查询过的数据, mybatis就会保存在一个缓存中(Map);下次获取直接从缓存中拿



5.3. 缓存-一级缓存失效的几种情况以及观察原理(271集)

一级缓存失效的几种情况

一级缓存是SqlSession级别的缓存

1.不同的sqlSession,使用不同的一级缓存;

只有在同一个sqlSession期间查询到的数据会保存到那个sqlSession的缓存中;下次使用这个sqlSession查询会从缓存中拿

2.同一个方法,不同的参数,由于可能之前没查询过,所以还会发新的sql

3.在sqlSession期间执行上任何一次增删改操作,增删改操作会把缓存清空

4.手动清空缓存


```
@Override
public void putObject(Object key, Object value) {
    cache.put(key, value);
}
// cache= HashMap<K,V> (id=79)
//   > entrySet= HashMap$EntrySet (id=84)
//   > keySet= null
//   > loadFactor= 0.75
//   > modCount= 3
//   > size= 1
//   > table= HashMap$Node<K,V>[16] (id=88)
//   > threshold= 12
//   > values= null
G 06 {-2115931584:3013824785:com.atguigu.dao.TeacherDao.getTeacherById:0:2147483647:select
G 06      where id=?:1:development=[Teacher [id=1, name=hahaha, course=语文, address=宝安
G 06
```

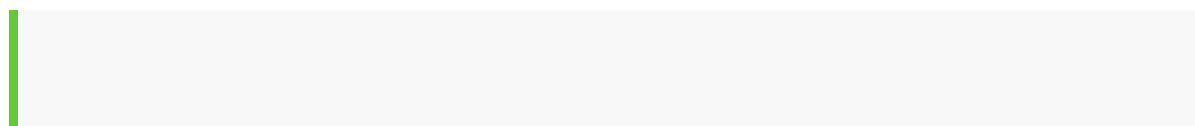
5.4. 缓存-二级缓存体验(272集)

二级缓存:

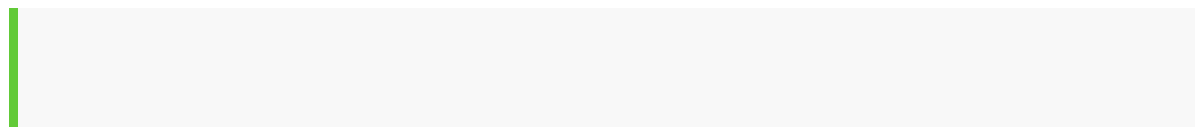
一级缓存:SqlSession关闭或者提交以后,一级缓存的数据就会放在二级缓存中;mybatis默认没有使用的;配置;

二级缓存的使用:

1.全局配置开启二级缓存



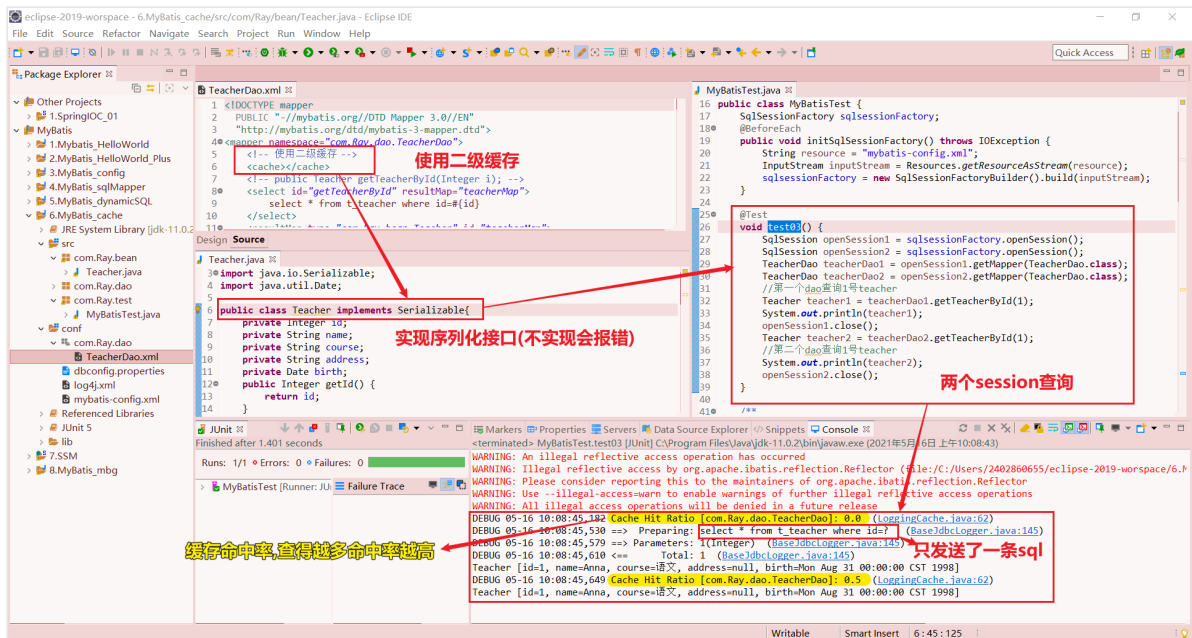
2.配置某个dao.xml文件,让其使用



```
MyBatisTest.java  Cache.class  mybatis-config.xml  TeacherDao.xml

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE configuration
3 PUBLIC "-//mybatis.org/DTD Config 3.0//EN"
4 "http://mybatis.org/dtd/mybatis-3-config.dtd">
5 <configuration>
6 <properties resource="dbconfig.properties"></properties>
7 <settings>
8 <!-- 开启延迟加载开关 -->
9 <setting name="LazyLoadingEnabled" value="true"/>
10 <!-- 开启属性按需加载 -->
11 <setting name="aggressiveLazyLoading" value="false"/>
12 <!-- 开启全局缓存开关 -->
13 <setting name="cacheEnabled" value="true"/>
14 </settings>
15 <environments default="development">
16 <environment id="development">
17 <transactionManager type="JDBC"/>
18 <dataSource type="POOLED">
19 <!-- ${}取出配置文件的值,这里不是EL表达式,凑巧一样罢了 -->
20 <property name="driver" value="${driverClass}"/>
21 <property name="url" value="${jdbcurl}"/>
22 </dataSource>
</environment>
</environments>
<sqlSessionFactory type="org.apache.ibatis.session.DefaultSqlSessionFactory">
</sqlSessionFactory>
</configuration>

1 <!DOCTYPE mapper
2 PUBLIC "-//mybatis.org/DTD Mapper 3.0//EN"
3 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
4 <mapper namespace="com.Ray.dao.TeacherDao">
5 <!-- 使用二级缓存 -->
6 <cache/>
7 <!-- public Teacher getTeacherById(Integer i); -->
8 <select id="getTeacherById" resultMap="teacherMap">
9 <select * from t_teacher where id=#{id}>
10 </select>
11 <resultMap type="com.Ray.bean.Teacher" id="teacherMap">
12 <id property="id" column="id"/>
13 <result property="address" column="address"/>
14 <result property="birth" column="birth_date"/>
15 <result property="course" column="class_name"/>
16 <result property="name" column="teacherName"/>
17 </resultMap>
18 <!-- public List<Teacher> getTeacherByCondition(Teacher teacher); -->
19 <select id="getTeacherByCondition" resultMap="teacherMap">
20 <select * from t_teacher
21 <!-- test="" 编写判断条件
22 <id>#{id} 取外传入的javaBean属性中的id值,判断是否不为空 -->
```



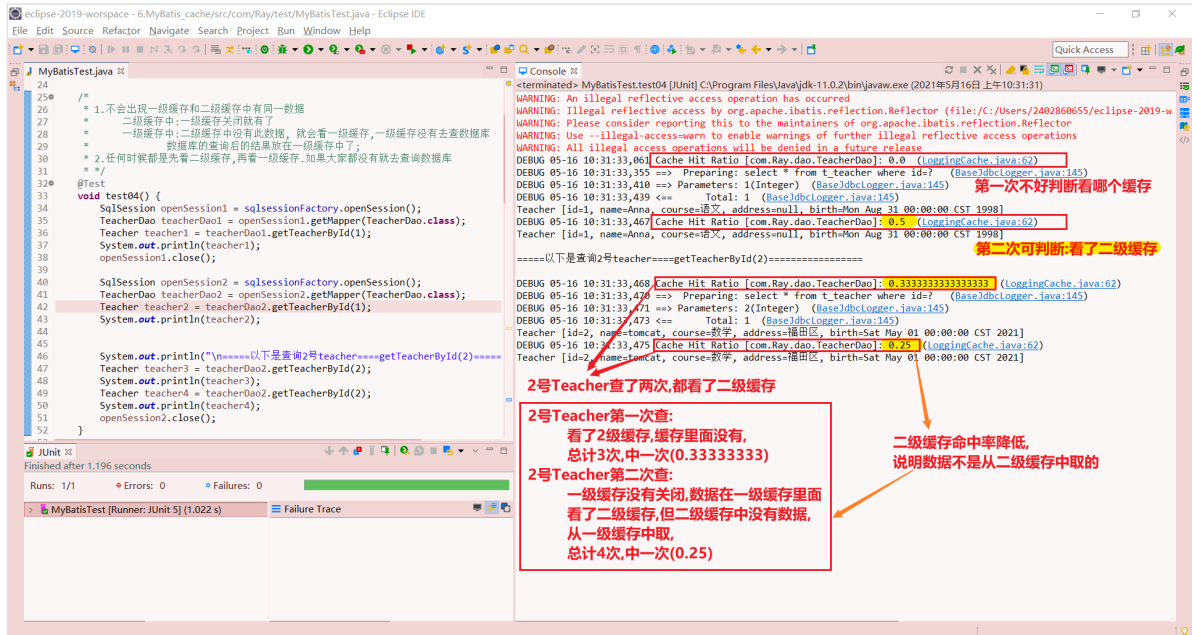
5.5. 缓存-缓存的查询顺序(273集)

1. 不会出现一级缓存和二级缓存中有同一数据

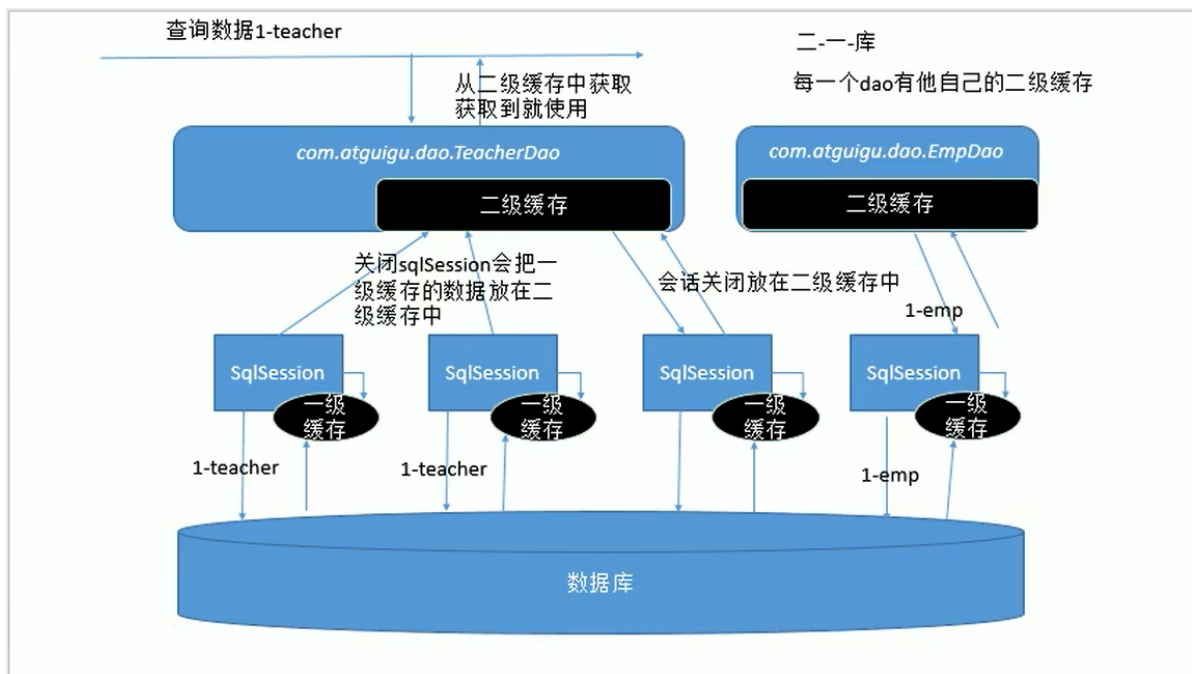
二级缓存中: 一级缓存关闭就有了

一级缓存中: 二级缓存中没有此数据, 就会看一级缓存, 一级缓存没有去查数据库
数据库的查询后的结果放在一级缓存中了;

2. 任何时候都是先看二级缓存, 再看一级缓存. 如果大家都没有就去查询数据库

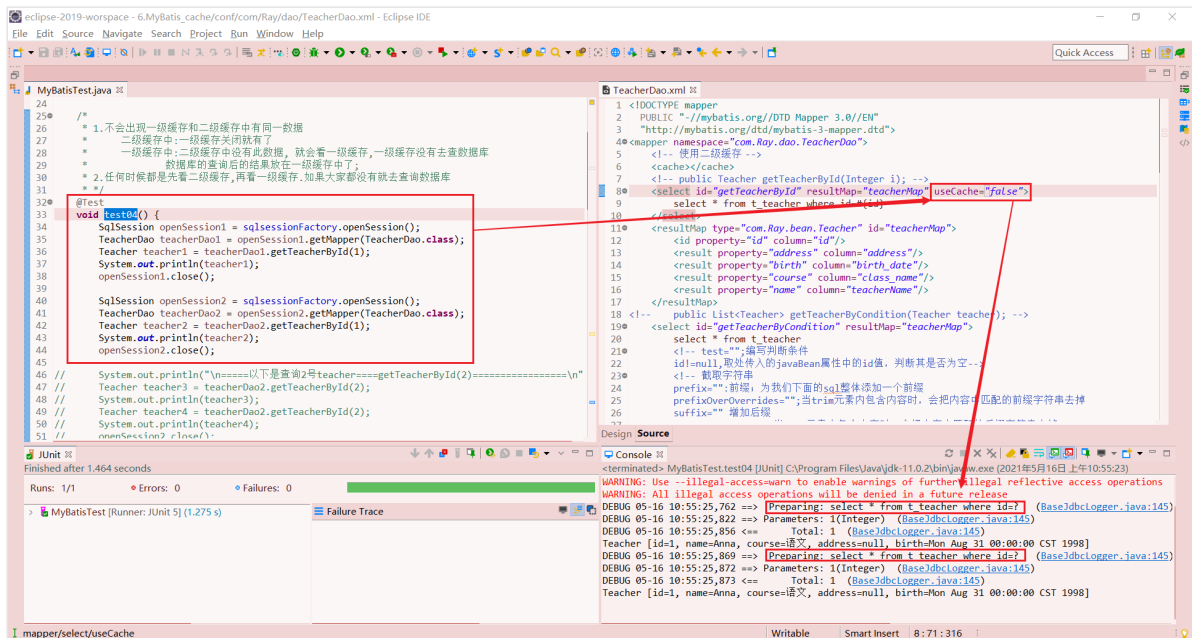


5.6. 缓存-缓存原理(274集)

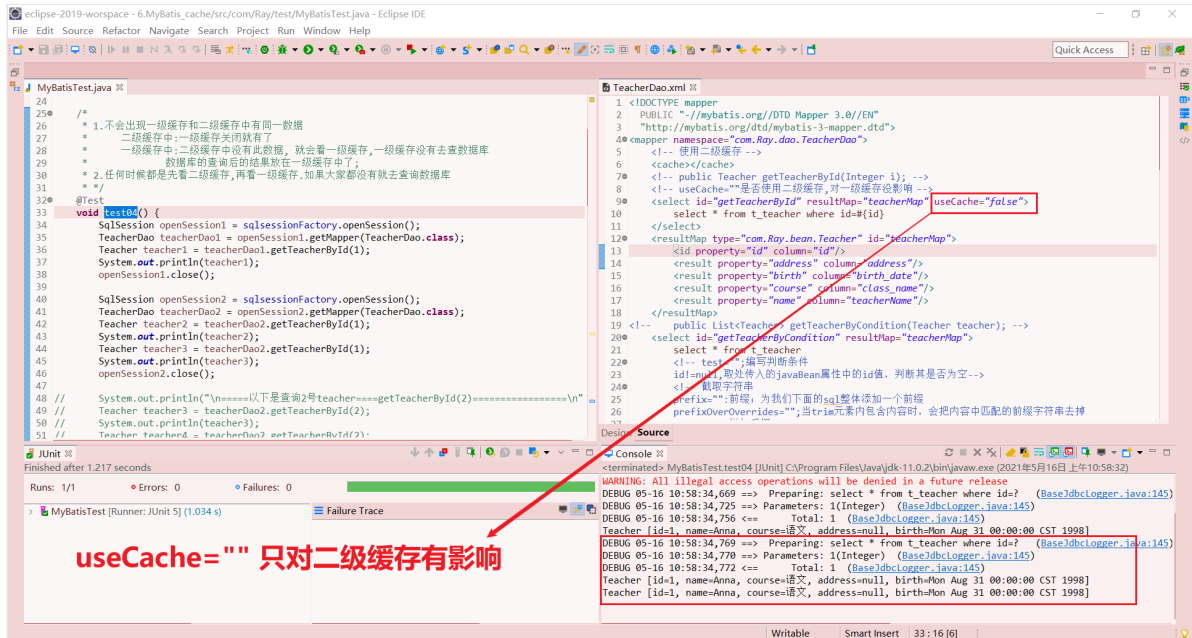


5.7. 缓存-缓存有关的属性(275集)

- 1、全局setting的cacheEnable：配置二级缓存的开关。一级缓存一直是打开的。
- 2、select标签的useCache属性：配置这个select是否使用二级缓存。一级缓存一直是使用的
- 3、sql标签的flushCache属性：增删改默认flushCache=true。sql执行以后，会同时清空一级和二级缓存。查询默认flushCache=false。
- 4、sqlSession.clearCache()：只是用来清除一级缓存。
- 5、当在某一个作用域 (一级缓存Session/二级缓存Namespaces) 进行了 C/U/D 操作后，默认该作用域下所有 select 中的缓存将被clear。



useCache=""是否使用二级缓存,对一级缓存没影响 默认是true



5.8. 缓存-整合第三方缓存(276集)

MyBatis开放了Cache接口

整合ehcache;ehcache非常专业的java进程内的缓存框架

1. 导包

```
ehcache-core-2.6.8.jar(ehcache核心包)
mybatis-ehcache-1.0.3.jar(ehcache与mybatis整合包,可以不写Cache的实现类)
slf4j-api-1.7.21.jar
slf4j-log4j12-1.7.21.jar
```

2. ehcache要工作有一个配置文件;

文件名叫ehcache.xml;放在类路径的根目录下

3. 在mapper.xml中配置使用自定义的缓存

```
1 <cache type="org.mybatis.caches.ehcache.EhcacheCache"></cache>
```

4.别的dao还要用

- 1 <!--和别的dao公用一块缓存-->
- 2 <cache namespace="com.Ray.dao.TeacherDao"></cache>

The screenshot shows the Eclipse IDE with the following components:

- Teacher.java:** A class with methods for getting and setting teacher information. The `getId()` method is highlighted with a red box.
- MyBatisTest.java:** A test class that uses the `TeacherDao` to get and set teacher information. The `test04()` method is highlighted with a red box.
- ehcache.xml:** A configuration file for Ehcache. The `<diskStore path="C:\44\ehcache" />` and `<defaultCache maxElementsInMemory="1" maxElementsOnDisk="10000000" eternal="false" />` elements are highlighted with red boxes.
- TeacherDao.xml:** A MyBatis configuration file. The `<cache type="org.mybatis.caches.ehcache.EhcacheCache"></cache>` element is highlighted with a red box.
- Console:** Shows the execution of the test method and the initialization of the cache. The output includes messages like "DEBUG 05-16 11:20:16,104 Failed to delete file" and "DEBUG 05-16 11:20:16,126 Initialised cache: com.Ray.dao.TeacherDao (Cache.java:116)".
- File Explorer:** Shows the file structure of the project, including the `ehcache` directory.